



Contents lists available at ScienceDirect

Data & Knowledge Engineering

journal homepage: www.elsevier.com/locate/datak

Discovering outlying attributes of outliers in data streams

Egawati Panjei^{a,b,*}, Le Gruenwald^a^a School of Computer Science, The University of Oklahoma, Devon Energy Hall 110 W. Boyd St., Norman, OK, 73069, United States^b Department of Computer Science, Lyle College of Engineering, Southern Methodist University, 3145 Dyer Street, Suite 445, Dallas, TX, 75206, United States

ARTICLE INFO

Keywords:

Outlying attributes
Anomaly explanation
Data streams

ABSTRACT

Data streams, continuous sequences of timestamped data points, necessitate real-time monitoring due to their time-sensitive nature. In various data stream applications, such as network security and credit card transaction monitoring, real-time detection of outliers is crucial, as these outliers often signify potential threats. Equally important is the real-time explanation of outliers, enabling users to glean insights and thereby shorten their investigation time. The investigation time for outliers is closely tied to their number of attributes, making it essential to provide explanations that detail which attributes are responsible for the abnormality of a data point, referred to as outlying attributes. However, the unbounded volume of data and concept drift of data streams pose challenges for discovering the outlying attributes of outliers in real time. In response, in this paper we propose EXOS, an algorithm designed for discovering the outlying attributes of multi-dimensional outliers in data streams. EXOS leverages cross-correlations among data streams, accommodates varying data stream schemas and arrival rates, and effectively addresses challenges related to the unbounded volume of data and concept drift. The algorithm is model-agnostic for point outlier detection and provides real-time explanations based on the local context of the outlier, derived from time-based tumbling windows. The paper provides a complexity analysis of EXOS and an experimental analysis comparing EXOS with existing algorithms. The evaluation includes an assessment of performance on both real-world and synthetic datasets in terms of average precision, recall, F1-score, and explanation time. The evaluation results show that, on average, EXOS achieves a 45.6% better F1 Score and is 7.3 times lower in explanation time compared to existing outlying attribute algorithms.

1. Introduction

Data streams are infinite sequences of data points, each timestamped (referred to as the notion of time) [1–3]. Unlike regular or static data, data streams are time-sensitive and demand real-time monitoring. For instance, in the context of network security monitoring, a sudden surge in network traffic could suggest a malware attack or compromised system [4,5]. It is imperative to process and analyze network traffic patterns in real-time, enabling the security team to identify potential security threats as they occur and to prevent or minimize damage to the network. Another illustration pertains to credit card transaction monitoring [6–8], where the system must promptly recognize unusual credit card activity to uphold customer trust and safety.

In streaming environments, data flows continuously from single or multiple sources, resulting in an unbounded volume of data for analysis. Data streams are thus said to have the notion of infinity [2]. The practical constraints such as memory limitations

* Corresponding author at: Department of Computer Science, Lyle College of Engineering, Southern Methodist University, 3145 Dyer Street, Suite 445, Dallas, TX, 75206, United States.

E-mail addresses: egawati.panjei@ou.edu, epanjei@smu.edu (E. Panjei), ggruenwald@ou.edu (L. Gruenwald).

<https://doi.org/10.1016/j.datak.2024.102349>

Received 30 January 2024; Received in revised form 15 August 2024; Accepted 21 August 2024

Available online 30 August 2024

0169-023X/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

render it unfeasible to store this infinite amount of data in memory for processing purposes [9]. As data points arrive from various sources, the arrival rate of each data stream can vary [2]. For instance, one source may omit data points every second while others may exhibit a different frequency such as every minute or every other minute.

Data stream sources can produce multi-dimensional data points, with the type and number of attributes potentially varying across streams [2]. Moreover, these streams may exhibit cross-correlation, signifying the attributes in one stream correlate with those in other streams [10,11]. For instance, measurements from a weather sensor in an area correlate with measurements from other sensors in neighboring areas. Additionally, data distribution in a stream can change over time, a phenomenon known as concept drift [12–16]. For example, weather patterns can change, causing shifts in temperature, precipitation, humidity, and other meteorological attributes.

Outliers or anomalies are “patterns in data that do not conform to a well-defined notion of normal behavior” [17]. Detecting outliers in data streams is incredibly important as they often indicate problems that require further exploration. For example, detected outliers in sensor data could point towards equipment failure, while spotting them in financial transactions might suggest fraudulent activities. After the outliers are detected, it falls upon users to examine the identified outliers and determine what actions to take [18].

Depending on applications, some detected outliers can demand quicker actions than others. For example, in Structural Health Monitoring (SHM) [19–21], outliers indicate infrastructure damage or sensor malfunctions. The former requires a faster response than the latter to prevent a disastrous event. To determine whether an outlier falls into the former or the latter case, engineers need to verify them. For this kind of critical application, the investigation should be done fast. However, users’ effort to investigate an outlier roughly corresponds to the number of attributes associated with the outlier [22]. The investigation takes time, but it can be done faster if each detected outlier is presented with explanations [18]. In [23], the outlier detection algorithm uses features extracted from raw sensor signals to determine whether any subsequent data point is a healthy signal or an outlier. If each outlier has an explanation as to which features are responsible for the detection of the outlier, engineers can verify the finding faster and take further action. This explanation is also known as outlying attributes.

Outlying attributes of an outlier refer to the subset of attributes responsible for the abnormality of the outlier. Some techniques such as [22,24–29] find the outlying attributes in static data where the notion of time of the data is not considered. They require multiple passes on the dataset which are not suitable to be applied on real-time data streams that are bound to memory constraints as data arrive continuously. In addition, they assume the data distribution is fixed, which is not the case with many stream applications. There exist outliers’ outlying attributes techniques that are designed for data stream applications [30–33]. They deal with the notion of infinity, yet some do not deal with the concept drift. Moreover, none of these techniques consider the cross-correlation among data streams. In data stream applications such as Wireless Sensor Networks (WSN), data collected on different nodes (sources) can exhibit a predictable relationship [10,11]. There exist not only dependencies among data generated by the same source but also dependencies among data generated by different sources.

To address the gap in the literature, we propose EXOS, an algorithm designed to generate outlying attributes for each outlier detected in single or multiple data streams. EXOS focuses on identifying outlying attributes of point anomalies. A point anomaly is a data point that is significantly different from other data points [2,17]. We assume that each data stream is associated with an independently running outlier detector. The multiple data streams can have identical or varying data arrival rates.

EXOS provides the outlier explanation in real time. To deal with the unbounded volume of data, EXOS generates outlying attributes using time-based tumbling [34] where a memory buffer is used to store a sequence of timestamped data of each data stream in the main memory. When a specified time period expires, i.e., when the window slides, all the data stored in the buffers are replaced with the new arriving sequence of data. Whenever a new window arrives, EXOS applies single-pass incremental computation on the data in the window, allowing it to process the data fast.

EXOS is a local neighborhood-based outlier explanation technique. It defines a local context for each outlier as a set of inlier neighbors and uses this to identify the outlier’s outlying attributes. The formation of the local context involves considering cross-correlations among data streams, with eigenvectors initially generated using offline data. To account for concept drift in data streams, the eigenvectors are updated whenever new sets of window slides through single-pass eigendecomposition.

This paper substantially extends our early conference paper [35]. Our contributions can be summarized as follows: (1) We propose EXOS, a real-time online outlier explanation algorithm identifying outlying attributes of multi-dimensional outliers in data streams. EXOS addresses the notion of time, notion of infinity, and concept drift characteristics of data streams. It can handle data streams with the same or different arrival rates.; (2) We provide a time and space complexity analysis of EXOS.; and (3) We conduct a comprehensive experimental analysis using real-world and synthetic datasets, comparing EXOS with existing algorithms in terms of average precision, recall, F1-score, and average explanation time.

The remainder of this paper is organized as follows: Section 2 provides the related work; Section 3 formally defines the problem with the underlying assumptions; Section 4 provides an overview of EXOS; Section 5 explains the algorithm in detail; Section 6 presents a time and space complexity of EXOS; Section 7 discusses experimental performance evaluation; and Section 8 provides the conclusions and future work.

2. Related work

There exist techniques that generate outlying attributes for outliers in static or non-stream data. Techniques like Refout [25], SFE [22], and Lookout [29] apply subspace search with heuristics to find a subset of attributes where a detected outlier has the highest outlier score. XPACS [27] uses a subspace clustering-based algorithm to find groups of outliers. It looks for candidate clusters

Table 1

Summary of the properties of the outliers' outlying attribute algorithms.

Algorithm	Data type	Notion of time	Notion of infinity	Concept drift	Cross-correlation	Concurrent	Non concurrent
Refout [25]	Static Data	No	No	No	No	No	No
SFE [22]	Static Data	No	No	No	No	No	No
Lookout [29]	Static Data	No	No	No	No	No	No
LODI [24]	Static Data	No	No	No	No	No	No
Micenková et al. [26]	Static Data	No	No	No	No	No	No
COIN [28]	Static Data	No	No	No	No	No	No
EXAD [32]	Data Stream	Yes	Yes	No	No	Independent	Independent
MICOES [33]	Data Stream	Yes	Yes	Yes	No	Independent	Independent
MacroBase [31,41]	Data Stream	Yes	Yes	Yes	No	Independent	Independent
EXstream [30,41]	Data Stream	Yes	Yes	Yes	No	Independent	Independent
EXOS (our algorithm)	Data Stream	Yes	Yes	Yes	Yes	Yes	Yes

that satisfy the minimum number of outliers threshold from one-dimensional subspace to l -dimensional subspace. It then refines the candidate clusters by keeping the outliers already in the clusters, removing the inliers included in the clusters, and adding the outliers not yet in the clusters. Each cluster is denoted as a conjunction of l attribute rules; thus, a cluster represents the outlying attributes of the outliers in it.

Techniques like LODI [24,26], and COIN [28] utilize inlier neighbors of an outlier to find its outlying attributes. They use a linear classifier to find a decision boundary between inliers and outlier classes. Attributes whose corresponding weights in the hyperplanes are higher than a threshold are considered outlying attributes. Kopp et al. [36] propose sapling random forests (SRF), an ensemble of specifically trained binary classification trees to explain outliers. Building an SRF requires sets of outliers and inliers. SRF produces a conjunctive normal form that consists of the most discriminative attributes of outliers and thus, denotes the outlying attributes. Takeishi [37] uses Shapley values of PCA reconstruction error to explain outliers detected using PCA. For each outlier, a Monte Carlo approximation is applied to find the Shapley value of each attribute. Outlying attributes are the attributes whose Shapley values are higher than a threshold.

In data streams, each data point is associated with a time stamp, either implicit or explicit, reflecting the notion of time. Unlike static data, data streams have an unbounded volume, often referred to as the notion of infinity [2,38]. All the aforementioned outlying attribute techniques are designed for static or non-stream data. They do not consider the data points' notion of time and require multiple passes on the dataset. Thus, they are not suitable for stream environments. Moreover, in a data stream, the underlying distribution or statistical properties of the data can change over time [14,39,40]. This phenomenon is called concept drift and it can occur because the data stream often comes from dynamic environments where the generating processes or data sources change. In addition, the data arrival rates of each data stream can be the same or different [2]. We refer to the data streams with the same arrival rates as concurrent data streams and the varying arrival rates as non-concurrent data streams.

There exist techniques proposed for data streams such as EXAD [32], EXstream [30,41], MICOES [33], MacroBase [31,41]. These techniques handle the notion of time and the notion of infinity of the data streams. EXAD uses a decision tree estimated from a neural network trained offline to generate outlying attributes of each detected outlier online. The outlying attributes of an outlier are denoted as a disjunctive normal form of features formed by a path starting from the root to the leaf where the outlier resides. Nevertheless, EXAD's decision tree is only good if the data distribution does not change. MICOES forms inlier and outlier classes to generate outlying attributes like LODI [24,26], and COIN [28], yet it utilizes incremental computation to generate outlying attributes. Its micro-cluster components adapt to changes in data distribution and thus allow the explanation to also adapt. MacroBase produces outlier explanations using a prefix tree of frequent itemsets updated periodically, allowing it to handle concept drifts. EXstream [30,41] generates outlier explanations based on the temporal context of the outliers, which is updated over time and thus captures the change in the data distribution. These existing data stream-based outlying explanation algorithms operate independently for each data stream. Thus, they do not need to worry whether the data streams are concurrent or not.

In Table 1, we present a summary of the properties of EXOS, and the existing algorithms designed for identifying outlying attributes of outliers. Notably, EXOS stands out as the sole algorithm tailored for data streams, simultaneously addressing the notion of time, the notion of infinity, concept drift, and cross-correlation while considering concurrent or non-concurrent data streams.

3. Preliminaries

This section formally defines the problem of finding outlying attributes of an outlier in data streams.

Definition 1 (Data Stream). A data stream denoted as S is an infinite sequence of data points $\{x_i | i \geq 0\}$. Each x_i is a tuple of length $d+1$ denoted as $x_i = a_1, a_2, \dots, a_d, t$ where d is the number of attributes, a_r is the value of the r -th attribute, and t is the associated timestamp when the tuple is recorded or collected.

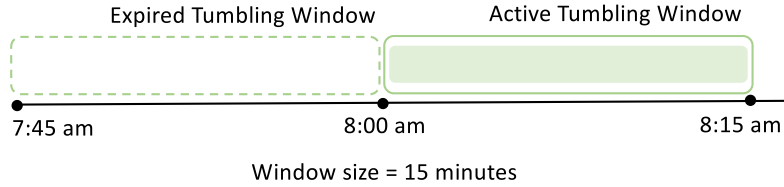


Fig. 1. Tumbling window's illustration.

We consider a set of m concurrent or non-concurrent data streams $\mathbb{S} = \{S_1, \dots, S_m\}$ where $m \geq 1$ and each number $1, \dots, m$ also represents a data stream ID. Data streams are concurrent when they have the same data arrival rates. Each data stream has a corresponding outlier detector. The i th data point in a data stream j (S_j), is denoted as x_i^j . We denote x_i^j as o_i^j when x_i^j is detected as an outlier. The attributes in S_j can differ from those in S_k . The total number of attributes in S_j is denoted as d_j . The set of attributes in S_j is denoted as $D_j = \{A_{j1}, A_{j2}, \dots, A_{jd_j}\}$. The total number of attributes of all m data streams is denoted as $D = d_1 + d_2 + \dots + d_m$.

Data streams arrive continuously and are inherently unbounded; hence, keeping all data in the memory for real-time processing is impossible. To address this challenge, EXOS employs a time-based tumbling window [34] for each data stream. A tumbling window (W) is a fixed size window that segment data streams into non-overlapping contiguous chunks of time (Definition 2). It serves as a buffer where all data points in the buffer remain available for a duration of time which is determined by the window size. When the current window expires, and a new window starts to receive new set of data points that arrive in the next period of time. To illustrate, Fig. 1 depicts expired and active tumbling windows of window size 15 min. The data points from the timestamps 7:45 am–8:00 am are held in the memory for 15 min, after which they are replaced by the data points from the timestamps 8:00 am–8:15 am.

Definition 2 (Tumbling Window). A tumbling window (W) can be mathematically defined by the following components:

1. window size, denoted as tws , which is a constant representing the length of each W
2. the start time of the p th W , denoted as T_p , which can be expressed as $T_p = T_0 + ptws$, where T_0 is the start time of the first or initial W . In this expression, $T_p = T_0 + ptws$ represents the timestamp after p times window size.
3. the end time of the p th W , denoted as T_{p+1} , which can be expressed as $T_{p+1} = T_p + tws$. In this expression, $T_p + tws$ means the timestamp after one window size.

A data point x with timestamp t belongs to the p th W if $T_p \leq t < T_{p+1}$. In other words, the p th tumbling window contains a sequence of data points whose timestamps are between T_p inclusively and T_{p+1} exclusively.

We refer multiple data streams with different data arrival rates as non-concurrent data streams. However, we assume that all data streams have the same tumbling window size, and they are all initialized at the same time.

The outlying attributes of an outlier are the subset of its attributes responsible for the abnormality of the outlier. They are formally defined in Definition 3.

Definition 3 (Outlying Attributes [18]). Given an outlier o , a set of d dimensions $D = \{A_1, A_2, \dots, A_d\}$ where $o \in A_1 \times A_2 \times \dots \times A_d$, an outlier attribute contribution score function $h : D \rightarrow \mathbb{R}$ that generates a real-value quantifying the contribution of each attribute to the abnormality of o , and an outlying contribution score threshold $\gamma \geq 0$, the outlying attributes of o is a subspace $F \subseteq D$ such that $\forall A_i \in F, h(A_i) > \gamma$.

Problem Definition: Given a set of m multi-dimensional data streams $\mathbb{S} = \{S_1, S_2, \dots, S_m\}$, an outlier o_i^j identified by an arbitrary outlier detector associated with the data stream S_j , and an outlying contribution score threshold $\gamma \geq 0$, the problem is to find all the outlying attributes of o_i^j accurately and efficiently.

We summarize the symbols we use in this paper in Table 2.

4. Overview of the EXOS algorithm

EXOS assumes each data stream is associated with an outlier detector and a tumbling window. Fig. 2 illustrates the interaction between the outlier detectors, tumbling windows, and EXOS. In this figure, data stream 1 (S_1) is associated with an outlier detector and a tumbling window (W), as are the other data streams. Recall that EXOS utilizes tumbling windows (Ws) to manage the continuous arrival and unbounded volume of data streams (the notion of infinity). EXOS assumes that all Ws across all data streams have the same window size. In each stream, arriving data points are consumed by the corresponding outlier detector and tumbling window. At the end of the active period of the Ws , EXOS absorbs the data points within the active Ws into its Estimator and Temporal Neighbor components. The Estimator component absorbs data points from the active Ws of all streams, while the Temporal Neighbor Component absorbs the data points from each active Ws of data stream j into its corresponding independent temporal neighbor function.

Table 2
List of Symbols.

Symbol	Interpretation
$\mathbb{S} = \{S_1, S_2, \dots, S_m\}$	A set of m data streams where each number of $\{1, 2, \dots, m\}$ also represents each stream's ID
m	Total number of data streams
S_j	A data stream j
W	Tumbling Window
W_j	An active tumbling window of a data stream j
tw_s	Tumbling window size
d_j	Total number of attributes in a data stream j
A	An attribute
$D_j = \{A_{j1}, A_{j2}, \dots, A_{jd_j}\}$	A set of attributes in a data stream j
D	Total number of attributes of all m data streams, $D = d_1 + d_2 + \dots + d_m$
σ_i^j	An outlier data point i in data stream j , $\sigma_i^j \in \mathbb{R}^{d_j}$
$\hat{\sigma}_i^j$	Estimated normal values of σ_i^j
$\mathbf{O}^j$	All outliers found in with timestamps within the start and end time of W_j . $\mathbf{O}^j$ is a matrix whose columns represent the attribute values and rows represent the outlier data points. The first and the last rows represent the oldest and the latest outliers in W_j respectively.
$\hat{\mathbf{O}}^j$	A matrix representing estimated normal values of $\mathbf{O}^j$
$\mathbf{Q}$	A matrix of size $D \times k$, representing k -eigenvectors,
$\mathbf{Q}_j$	A segment of $\mathbf{Q}$ that is associated with S_j . $\mathbf{Q}_j$ is a matrix of size $D \times k$.
$\mathbf{Z}$	A matrix of size $D \times k$, used to update $\mathbf{Q}$
$\mathbf{x}_{\text{aligned}}$	A tuple of D combined values of data points from all streams that share the same timestamp, $\mathbf{x}_{\text{aligned}} \in \mathbb{R}^D$
AlignedData	A key-value set where a key represents a timestamp, and its corresponding value represents $\mathbf{x}_{\text{aligned}}$
$\mathbb{C}_j = C_1, \dots, C_{l_j}$	A set of l_j clusters linked to data stream j
l_j	Number of clusters in a data stream j
num_points_{aj}	The number of data points absorbed by a cluster $C_a \in \mathbb{C}_j$
$\mathbf{centroid}_{aj}$	The centroid of a cluster $C_a \in \mathbb{C}_j$, $\mathbf{centroid}_{aj} \in \mathbb{R}^{d_j}$
Est	The estimator component of EXOS
Cf_j	An independent function j that clusters data in the active tumbling window at stream j
$\mathcal{G}_j$	An independent function j associated with stream j that forms a local context of each outlier based on the output of Est and Cf_j , and then computes the outlying contribution score of the outlier's attributes
N_j	The number of data points in the active tumbling window of data stream j (W_j)
$ \mathbf{O}^j $	The number of outliers in W_j
n_{samples}	The maximum number of auxiliary data points generated for the inlier or outlier classes
I	The maximum number of iterations required to initialize clusters of each stream
nepochs	The number of iterations to train a linear SVM using a gradient descent algorithm

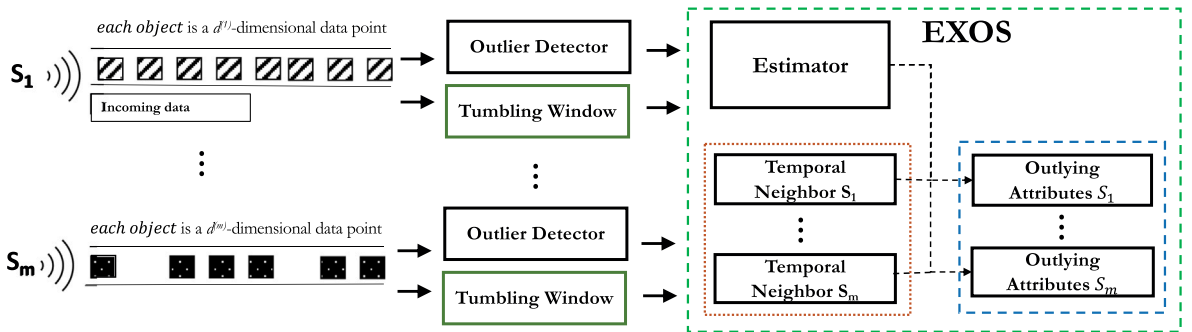


Fig. 2. The interaction between outlier detectors, tumbling windows, and EXOS.

While absorbing data points from the active W_s , EXOS will also check whether the outlier detectors identify any outliers during within the duration of the active W_s . If any outliers are detected, EXOS will activate its third component: Outlying Attributes

Generator. This component takes the outputs from the Estimator and Temporal Neighbor components as its inputs and proceeds to find the outlying attributes of each outlier detected in each stream.

Like COIN [28], EXOS generates the outlying attributes of each outlier based on its local neighborhood (local context). This approach allows EXOS to avoid the subspace search. Based on the definition that an outlier is a data point that is significantly different than the rest of the data points, COIN uses the closest normal data points (inliers) to an outlier to serve as the outlier's local context. However, its neighborhood computation requires storing the entire set of data points, which is impractical in a data stream environment. Additionally, the notion of time implies that neighborhood computation should consider the timestamp of the outlier. Thus, identifying the local context for each outlier in a stream setting is not straightforward.

To handle neighborhood computation in the stream setting, EXOS estimates the normal value of each outlier using a model that captures the correlation among the attributes within the data stream where the outlier is detected as well as across other data streams (cross-correlation). The model is updated periodically, specifically when EXOS absorbs new data points from active Ws, to ensure it captures changes in data distribution (concept drift). This estimation of the normal values of each outlier and the model update are performed by the Estimator component. While Estimator is doing its job, the other component of EXOS, the Temporal Neighbor, clusters data points from each data stream independently in an incremental way. The clusters do not retain the original data points but only maintain the summary or statistics of the absorbed data points.

The Outlying Attributes Generator component takes as input the estimated normal value of each outlier and statistics derived from clusters within the data streams. It operates with m parallel independent functions, each dedicated to a specific data stream. Each function j within this component is responsible for constructing a local context for each outlier from data stream j (S_j), using both the estimated normal value of the outlier and the clusters identified within S_j . Once the local context is established, the function determines the outlier's class. Subsequently, the Outlying Attributes Generator function trains a linear classifier to differentiate the local context from the outlier class. The weights assigned to each attribute in the classifier's decision boundary are then used to identify which attributes constitute outlying attributes of the outlier.

5. The EXOS algorithm

Algorithm Section 5 shows the EXOS algorithm. It consists of the initialization phase (Lines 1–2) and the Online Phase (Lines 3–24). The initialization phase consists of two tasks: (i) initializing eigenvectors $\mathbf{Q}$ and (ii) setting the list of the initial clusters' centroids in all streams (Line 2). EXOS makes use of the cross-correlation characteristics of data streams to determine the outlying attributes of outliers in each stream. It captures the cross-correlation by applying a single-pass incremental Principal Component Analysis (PCA)-based technique to the data in the active windows. This technique requires the initialization of k eigenvectors or principal components $\mathbf{Q}$ before it runs online.

We apply naive PCA [42] on offline data to generate the initial eigenvectors $\mathbf{Q}$ (Line 1). The offline data consist of initial data of all data streams. Recall that the total number of attributes of all data stream is D . The Naive PCA first obtains a $D \times D$ correlation matrix by calculating the correlation coefficients between each pair of attributes in the offline data. It then applies an eigenvalue decomposition to the correlation matrix. The decomposition process factorizes the correlation matrix into its eigenvalues and eigenvectors. Eigenvalues indicate the amount of variance explained by each principal component (or eigenvector). Larger eigenvalues correspond to components that capture more variance. Eigenvectors represents the directions in the original attribute space that capture maximum variance. Selecting k first principal components means selecting the first k eigenvectors associated with the k largest eigenvalues. To this end, the initial $\mathbf{Q}$ is a $D \times k$ eigenvectors matrix where $\mathbf{Q} \in \mathbb{R}^{D \times k}$, $k < D$.

Moreover, the offline data are also clustered using the k -means clustering algorithm [97] (Line 2). The clusters' centroids are stored in the AllClust data structure to be used by the Temporal Neighbor Clustering component. AllClust is a list of m sets of clusters of m data streams. Each cluster in the set of clusters is represented by its centroid and the number of data points it has. The Online Phase indicated in Lines 3–23. Before a new tumbling window starts, EXOS consumes data points from all m active tumbling windows and store it into a data structure called TWinS (Line 6). TWinS is a list of m groups of data points, where a group j contains data points from data stream j 's active tumbling window. Recall that all active tumbling of all data streams has the same tumbling window size, starting time, and ending time. EXOS also obtains all outliers detected during the period of all active tumbling windows and stores it in a data structure called AllOutliers (Line 7). AllOutliers is a key–value set, where a key represents a data stream's ID and its corresponding value represent a set of outliers whose timestamp within the duration of the active tumbling window of the particular stream ID.

Lines 8–19 show how the three key components of EXOS, (i) Estimator Est (Line 9), (ii) Temporal Neighbor Clustering (Line 10–13) and (iii) Outlying Attributes Generator (Lines 15–19), are connected. The Est component uses the eigenvectors $\mathbf{Q}$ to estimate the normal values of outliers. It incrementally updates $\mathbf{Q}$ whenever it absorbs data points from all active tumbling windows to ensure that the changes of data distribution, known as the concept drift, are reflected in $\mathbf{Q}$. Running concurrently with Est, the Temporal Neighbor component consists of m parallel clustering functions, $Cf_1, Cf_2, \dots, Cf_m$. Each Cf_j incrementally absorbs each data point in the active tumbling window of S_j (W_j) into a cluster by selecting the cluster whose centroid has the closest distance to the data point.

To deal with the issue of unbounded volume of data under the constraints of limited memory, EXOS does not keep the original data in the clusters. It only retains the centroids and the number of data points the clusters. The Outlying Attribute Generator component also features m parallel functions, $G_1, G_2, \dots, G_m$. Each G_j corresponds to a data stream j . It uses the estimated normal values of outliers from Est and the latest centroids from Cf_j to identify the outlying attributes of each outlier o_i^j detected in data stream j .

We now describe the three key components of EXOS.

Algorithm 1: EXOS ($\mathbb{S}$, $k, D, \gamma, ds_attrs, tws, init_data, n_clusters, outlier_queues$)
Input: $\mathbb{S}$ the set of m concurrent data streams, k the number of eigenvectors, D the total number of attributes, γ the outlying contribution threshold, ds_attrs the list of the attributes in each stream, tws the window size, $init_data$ the offline data, $n_clusters$ the list of number of clusters in each data stream, $outlier_queues$ the list of m queues storing the outliers of the m streams.
Output: F_j the set of outlying attributes of each outlier in each data stream j
<pre> ## Initialization 1 $\mathbf{Q} := \text{initialize_eigen_vectors} (init_data)$ 2 $AllClust := \text{initialize_set_of_clusters} (init_data, n_clusters)$ 3 $end_ts := \text{current_time} ()$ 4 $start_ts := end_ts - tws$ ## Start the Online Phase 5 while (true): ## repeat indefinitely 6 $TWinS = \text{get_active_tumbling_windows_data_points} (\mathbb{S}, start_ts, end_ts)$ 7 $AllOutliers = \text{get_outliers_from_each_stream} (outlier_queues, start_ts, end_ts)$ 8 do in parallel 9 ##The Estimator Component 10 $\mathbf{Q}, est_list := \text{Estimator} (TWinS, k, D, \mathbf{Q}, tws, AllOutliers, arrival_rates,$ 11 $ds_attrs)$ 12 ##The Temporal Neighbor Component 13 parallel for j in $[1: m]$ 14 $AllClust[j] := \text{TemporalNeighborClusteringAtS}_j (TWinS[j], AllClust[j],$ 15 $n_clusters[j])$ 16 end parallel for 17 end do in parallel 18 ## The Outlying Attribute Generator Component 19 parallel for j in $[1: m]$ 20 $\hat{\mathbf{O}}_j := est_list[j]$ ## get estimated normal values of the outliers in stream j 21 $\mathbf{O}_j := \text{get_outlier_values} (AllOutliers[j])$ ## get outlier values in stream j 22 $OutAttrListj := \text{OutlyingAttrGeneratorAtS}_j (\mathbf{O}_j, \hat{\mathbf{O}}_j, AllClust[j], \gamma, ds_attrs[j])$ 23 ## return the outlying attributes of the outliers in the current window of 24 data stream j 25 $yields(OutAttrListj)$ 26 end parallel for 27 while $(end_ts + tws > \text{current_time}())$: wait() end while 28 $end_ts = \text{current_time} (); start_ts = end_ts - tws$ 29 end while </pre>

Algorithm 1: EXOS.

5.1. The estimator component

The Estimator Component, denoted as Est, is responsible for approximating the normal value $\hat{o}_i^j$ of each outlier o_i^j in each tumbling window. This task is accomplished by leveraging the attribute correlations within the data stream S_j and considering the inter-attribute correlations across other data streams. To illustrate this process, imagine a scenario involving three-source data streams depicted in Fig. 3. In this scenario, data streams S_1, S_2 , and S_3 have three attributes (A_{11}, A_{12}, A_{13}), two attributes (A_{21}, A_{22}), and two attributes (A_{31}, A_{32}) attributes, respectively and their data points arrive every 1, 2, and 4 min respectively. The data are observed between the time frame of 8:00 am and 8:15 am, during which the outlier detector at S_1 identifies an outlier at 8:03 am. The outlier detected at 8:03 am is denoted as o_{t+2}^1 in Fig. 3.(a) and its normal value estimation is denoted as $\hat{o}_{t+2}^1$ in Fig. 3.(b).

In the quest to find the estimated normal values of outliers within an active tumbling window, each time an old tumbling window expires and a new one arrives; Est aligns the data from the active tumbling window of each stream according to their timestamp and arrival rate. For instance, in the previously described scenario, the data alignment results in a 7×15 matrix showcasing the data points from data streams S_1, S_2 , and S_3 between 8:01 am and 8:15 am as depicted in Fig. 4. The matrix rows represent the

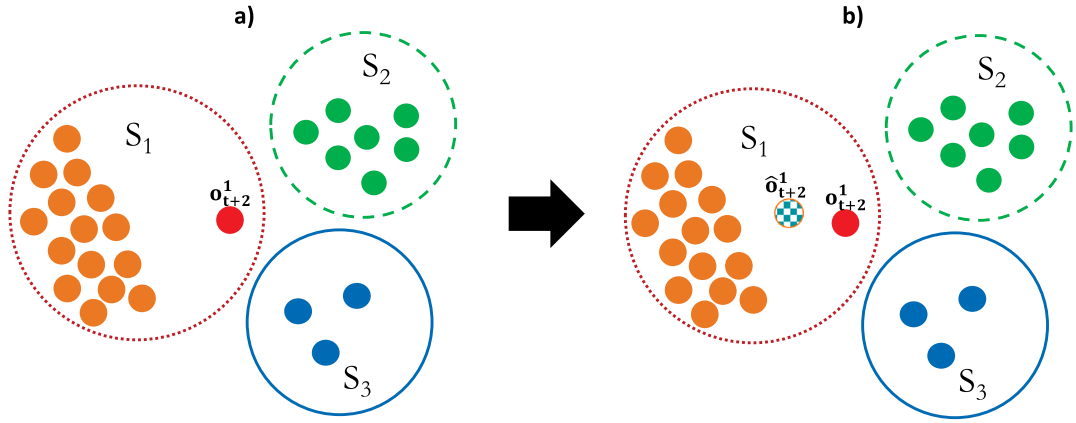


Fig. 3. Illustration of three data streams and their outliers.

	ts	ts+1	ts+2	ts+3	ts+4	ts+5	ts+6	ts+7	ts+8	ts+9	ts+10	ts+11	ts+12	ts+13	ts+14
S_1	A_{11}	□	□	□	□	□	□	□	□	□	□	□	□	□	□
	A_{12}	□	□	□	□	□	□	□	□	□	□	□	□	□	□
	A_{13}	□	□	□	□	□	□	□	□	□	□	□	□	□	□
S_2	A_{21}		□			□		□		□		□		□	
	A_{22}		□			□		□		□		□		□	
S_3	A_{31}			□				□				□			
	A_{32}			□				□				□			

Fig. 4. An example of data alignment results from three data streams of 3, 2, and 2 attributes in Fig. 3.

attributes from each stream while the matrix columns represent the timestamp steps of the active tumbling windows. The total number of the timestamp steps is determined by the data stream with the fastest arrival rate. In our previously described scenario, we have 15 timestamp steps from t to $t+14$ since S_1 's data points arrive every one minute. It is important to note that when dealing with a single data stream, the data alignment encompasses all data points within the active tumbling windows.

In the situation where the arrival rates among all data streams are similar or when dealing with a single stream, Est utilizes all the aligned data from the active tumbling windows as input to the PCA-based algorithm. However, when the data stream arrival rates differ, Est selectively employs the aligned data corresponding to the timestamp steps that contain the data points from all streams. For instance, in our scenario, Est selects the data points associated with the timestamp steps $t+3$, $t+7$, and $t+11$ as input to the PCA-based algorithm. Let us denote each selected aligned data point as $\mathbf{x}_{\text{aligned}} \in \mathbb{R}^D$. The vector representation of $\mathbf{x}$ is given by:

$$\mathbf{x}_{\text{aligned}} = \begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \\ a_{21} \\ a_{22} \\ a_{31} \\ a_{32} \end{bmatrix}$$

where a_{11} , a_{12} , a_{13} , a_{21} , a_{22} , a_{31} and a_{32} represent the values of attributes A_{11} , A_{12} , A_{13} , A_{21} , A_{22} , A_{31} , and A_{32} respectively.

Est uses a PCA-based approach which is primarily used to transform high-dimensional data into lower-dimensional space while preserving the most important information [43]. PCA achieves this by identifying orthogonal axes (also known as principal components or eigenvectors) along which the variance in the data is maximized. It focuses on capturing the variance in the data, with the first eigenvector explaining the most variance, the second explaining the second most, and so on. Each eigenvector is a linear combination of the attributes. In the process of identifying the eigenvectors, PCA takes into account the covariance between attributes. Attributes with higher covariances will have a stronger influence on the eigenvectors. In this sense, PCA can indirectly reveal the relationships among attributes, especially if there are strong correlations between them. High positive or negative correlations are reflected in the eigenvectors.

Whenever Est handles a new active tumbling window, its PCA part derives k eigenvectors (principal components) from the selected aligned data of the tumbling window. The continuous update on k eigenvectors allows Est to adapt to concept drift in the data streams. Arguably, Est can use the naive PCA to generate the eigenvectors in every window. However, the naive PCA ignores the data seen in the previous windows and requires multiple passes on the aligned data, which is not suitable to handle real-time continuous data streams with limited memory. Hence to update the k eigenvectors, we adopt DBPCA, a single pass eigendecomposition algorithm described in [44]. Our adaption of DBPCA is described in the following paragraph and also shown in Algorithm 2 Lines 5–8.

Let the k eigenvectors be $\mathbf{Q}$, the matrix of size $D \times k$ where $\mathbf{Q} \in \mathbb{R}^{D \times k}$, $k < D$. From our previously described scenario and given $k = 1$, $\mathbf{Q}$ then contain one eigenvector as follow:

$$\mathbf{Q} = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \end{bmatrix}$$

The values $e_1, e_2, e_3, e_4, e_5, e_6$ and e_7 correspond to the attributes $A_{11}, A_{12}, A_{13}, A_{21}, A_{22}, A_{31}$, and A_{32} respectively. When a new tumbling window arrives, the DBPCA part of Est initializes a zero matrix $\mathbf{Z}$ of size $D \times k$ and then for each selected aligned data point $x \in \mathbb{R}^D$, it incrementally updates $\mathbf{Z}$ using the following equation:

$$\mathbf{Z} = \mathbf{Z} + \frac{1}{N} \mathbf{x}_{\text{aligned}} \mathbf{x}_{\text{aligned}}^T \mathbf{Q}$$

where N is the number of selected aligned data points, $\mathbf{x}_{\text{aligned}}^T$ is the transpose of $\mathbf{x}_{\text{aligned}}$ and $\mathbf{Q}$ is the matrix containing the k eigenvectors from the previous window. After conducting a single pass iteration over all selected aligned data points to update $\mathbf{Z}$, Est then computes the new $\mathbf{Q}$ using the Q-R decomposition of $\mathbf{Z}$.

After obtaining the updated $\mathbf{Q}$, Est duplicates it and retains the new $\mathbf{Q}$ in memory to serve as the initial $\mathbf{Q}$ for the subsequent window. The duplicated $\mathbf{Q}$ is then sliced into m distinct segments: $\mathbf{Q}_1, \dots, \mathbf{Q}_j, \dots, \mathbf{Q}_m$. In our example, the copied $\mathbf{Q}$ is segmented into three parts: $\mathbf{Q}_1, \mathbf{Q}_2$, and $\mathbf{Q}_3$. Each segment corresponds to a specific stream as follows:

$$\mathbf{Q}_1 = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix}, \mathbf{Q}_2 = \begin{bmatrix} e_4 \\ e_5 \end{bmatrix}, \text{ and } \mathbf{Q}_3 = \begin{bmatrix} e_6 \\ e_7 \end{bmatrix}.$$

Each $\mathbf{Q}_j$ is then utilized to estimate the normal values of an outlier in S_j using the following matrix operation: $\hat{o}_i^j = \mathbf{o}_i^j \mathbf{Q}_j \mathbf{Q}_j^T$

In our previous example, the estimated normal value of the outlier detected at 8:03 am in data stream S1 is calculated as follows:

$$\hat{o}_{i+2}^1 = \mathbf{o}_{i+2}^1 \mathbf{Q}_1 \mathbf{Q}_1^T = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}$$

Furthermore, when there is more than one outlier in a tumbling window of a data stream S_j , the outliers can be consolidated into a matrix $\mathbf{O}^j \in \mathbb{R}^{\#o^j \times d_j}$ where $\#o^j$ is the total number of outliers in S_j and d_j is the number of attributes in S_j . The estimated normal values of the outliers in the window then become:

$$\hat{\mathbf{O}}^j = \mathbf{O}^j \mathbf{Q}_j \mathbf{Q}_j^T$$

Algorithm 2 summarizes how Est works. In Line 1, the algorithm computes the total number of timestamp steps required to align the data points from the data streams' active windows. It then processes the active windows to get the selected aligned data points (AlignedData) (Line 2). The AlignedData data structure of our example in Fig. 3 is depicted in Fig. 5.

Lines 4–9 are the DBPCA approach responsible for updating the eigenvectors matrix $\mathbf{Q}$ based on the combined data points from all streams in AlignedData. The DBPCA goes through each $\mathbf{x}_{\text{aligned}} \in \mathbb{R}^D$ in AlignedData to update $\mathbf{Z}$ (Line 7) and then computes the new $\mathbf{Q}$ (Line 9). The copied new $\mathbf{Q}$ is sliced into m segments (Line 10) and each segment of $\mathbf{Q}$ is then used to estimate the normal values of the outliers in its corresponding data stream (Lines 11–17). The algorithm returns the new eigenvectors matrix $\mathbf{Q}$ and the list of the estimated normal values of the outliers.

5.2. The temporal neighbor clustering component

The temporal neighbor clustering component is a set of m independent functions $Cf_1, Cf_2, \dots, Cf_m$ that runs in parallel with the estimator component. Each Cf_j receives data points from the active tumbling window of data stream j (W_j) and integrates each data point into a cluster. In regular or static dataset, the neighborhood computation of an outlier requires all data points to be available. However, this approach is impractical for data streams, where data points continuously arrive, making it impossible to store all data points in the memory. To address this issue, the temporal neighbor clustering function for data stream j (Cf_j) has the

Algorithm 2: Estimator (TWinS, k, D, Q , tws, AllOutliers, arrival_rates, ds_attrs)
Input: <i>TWinS</i> A list of m groups of data points, where a group j obtains data points from data stream j 's active tumbling window, k the number of eigenvectors, D the total number of attributes, Q the previous eigenvectors, tws the tumbling window size, AllOutliers the key-value set, where a key represents a data stream's ID and its corresponding value represents a set of outliers whose timestamp with the duration of the active tumbling window of the particular stream ID, <i>arrival_rates</i> the list containing the arrival rates of all data streams, <i>ds_attrs</i> the list containing the numbers of attributes of all data streams
Output: updated eigenvectors Q , list of estimated normal values of outliers <i>est_list</i>
<pre> ## compute the total number of time stamp steps for aligned data 1 n_ts_step = tws / min(arrival_rates) ## align data points from all data streams' active tumbling windows 2 AlignedData = get_selected_aligned_data (TWinS, n_ts_step) 3 $\bar{N} = \text{AlignedData}$ ## get the number of selected timestamp steps 4 Initialize a zero matrix Z of size $D \times k$ 5 for each ts_step in AlignedData do 6 $\mathbf{x}_{\text{aligned}} = \text{AlignedData}[\text{ts_step}]$ 7 $\mathbf{Z} = \mathbf{Z} + \frac{1}{\bar{N}} \mathbf{x}_{\text{aligned}} \mathbf{x}_{\text{aligned}}^T \mathbf{Q}$ 8 end for 9 Q = QR-decomposition of Z ## copy and divide eigenvectors into m segments 10 Q_segments = slice_eigenvectors(Q, ds_attrs) 11 Set est_list as a list of length TWinS 12 parallel for $1 \leq j \leq \text{TWinS}$ do: 13 $\mathbf{Q}_j = \text{Q_segments}[j]$ 14 $\mathbf{O}^j = \text{get_outlier_matrix}(\text{AllOutliers}[j])$ 15 $\hat{\mathbf{O}}^j = \mathbf{O}^j \mathbf{Q}_j \mathbf{Q}_j^T$ 16 est_list[j] = $\hat{\mathbf{O}}^j$ 17 end parallel for 18 return Q, est_list </pre>

Algorithm 2: Estimator.

role to maintain statistics of data points without storing them all. These cluster statistics are later used to form a local context for each outlier detected in the same tumbling window. By using cluster statistics updated from the same tumbling window in which the outlier is detected, we ensure that the outlier's local context (the selected neighborhood of the outlier) is both relevant and timely.

We now describe how the temporal neighbor clustering function for data stream j (Cf_j) works. As depicted in Fig. 6, each data point from the active tumbling window of data stream j (W_j) is grouped into a set of clusters $\mathbb{C}_j = C_{1j}, \dots, C_{l_j}$ such that $C_{aj} \cap C_{bj} = \emptyset$ for all $a \neq b$ and the number data points in W_j = the number of data points that all clusters $\bigcup_{a=1}^{l_j} C_{aj}$ absorb. $\mathbb{C}_j$ acts as the temporal neighbors of outliers that belongs in W_j . The number of clusters in $\mathbb{C}_j$ is denoted as l_j . Due to the unbounded amount of data points in data streams and the limited memory, the clusters cannot retain the data points in the memory. Hence, each cluster $C_{aj} \in \mathbb{C}_j$ only keeps the number of points in the cluster (num_points_{aj}) and C_{aj} , and the centroid of the cluster ($\mathbf{centroid}_{aj} \in \mathbb{R}^{d_j}$).

The distance between a data point $\mathbf{x} \in TW_j$ and a cluster's centroid $\mathbf{centroid}_{aj}$ can be represented as a distance function $d(\mathbf{x}, \mathbf{centroid}_{aj})$. By excluding the timestamp from $\mathbf{x}$, the distance function denoted as $d(\mathbf{x}, \mathbf{centroid}_{aj})$ is formulated as

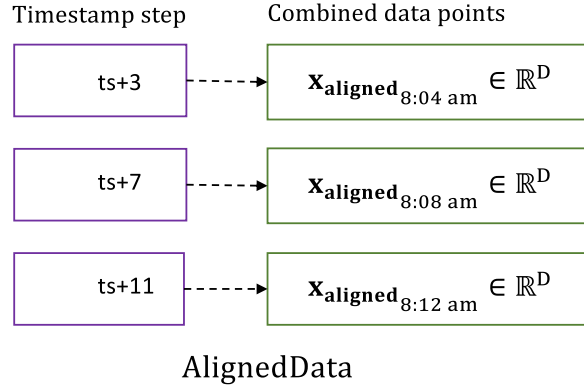


Fig. 5. Illustration of the AlignedData data structure based on the example in Fig. 3.

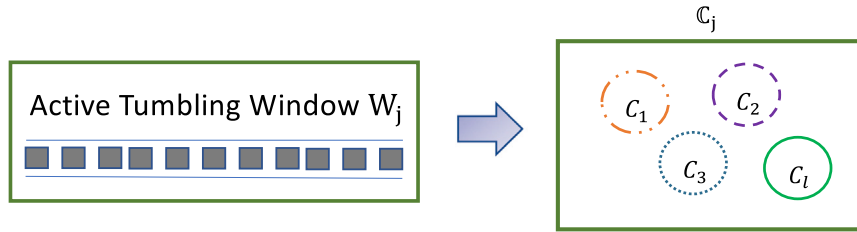


Fig. 6. Grouping data points in the active tumbling window W_j into a set of clusters.

$d(\mathbf{x}, \mathbf{centroid}_{aj}) = \mathbf{x} - \mathbf{centroid}_{aj}$. The data point $\mathbf{x}$ is assigned to a cluster $C_{aj} \in \mathbb{C}_j$ such that the distance between $\mathbf{x}$ and $\mathbf{centroid}_{aj}$ is minimized.

Algorithm 3 delineates the operations within an independent temporal neighbor clustering function Cf_j . This function takes the active tumbling window of data stream j (W_j) and the initial centroids of all clusters in the window as inputs. It starts by setting each cluster's centroid to its corresponding initial centroid and setting the total number of data points in each cluster to one (Lines 1–3). Given the continuous arrival of data points necessitating a single-pass computation, forming clusters of data points in the window is done incrementally. The sequential k-means algorithm [100] is employed to update the set of clusters $\mathbb{C}_j$ (Lines 4–8). The algorithm iterates through each data point from W_j . For each iteration, it assigns a data point $\mathbf{x}$ to the nearest cluster (Line 5). Subsequently, it updates the properties of the assigned cluster, by incrementing its total number of data points num_points_{aj} by one (Line 6) and updating its $\mathbf{centroid}_{aj}$ (Line 7) using the following formula:

$$C_{aj}.\mathbf{centroid}_{aj} = C_{aj}.\mathbf{centroid}_{aj} + \frac{\mathbf{x} - C_{aj}.\mathbf{centroid}_{aj}}{C_{aj}.num_points_{aj}}$$

When the algorithm finishes iterating over each data point once, it returns the set of clusters $\mathbb{C}_j$.

Incorporating the sequential k-means algorithm, allows each independent function Cf_j of the Temporal Neighbor Component to conduct a single pass incremental computation. As we can see, each data point in the active tumbling window of data stream j is processed one at a time. One could argue that the process of assigning a data point into a cluster in $\mathbb{C}_j$ in Line 5, requires l_j distance computation between the data point and the centroids of l_j clusters that can be done in parallel. Thus, the process can be seen as a single pass. Furthermore, unlike traditional k-means algorithm which require multiple set over the dataset to update the centroids of the clusters, the sequential k-means algorithm updates the centroids immediately after processing each individual data point. By processing each data point individually and updating the centroids immediately, the sequential k-means algorithm avoids the need to store the entire dataset in the memory.

5.3. The outlying attribute generator component

The outlying attribute generator component is a set of m independent functions $\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_m$. Each $\mathcal{G}_j$ corresponds to a data stream j and receives the inputs produced by the estimator component Est and the temporal neighbor component Cf_j . It is activated whenever a tumbling window associated with data stream j (W_j) contains one or more outliers. Est provided the estimated normal value of each outlier detected in W_j while Cf_j provided the clusters of data points from W_j . Together these inputs are used by $\mathcal{G}_j$ to form a local context of each outlier detected in W_j , generating outlying attributes of each outlier.

Algorithm 3: TemporalNeighborClusteringAtS_j(W_j, init_centroids, l_j)
Input: W _j the active window of S _j , init_centroids the list of initial centroids of clusters, l _j the number of clusters in data stream j
Output: C _j the set of l _j clusters of the active window
<pre> 1 set C_j as a set of l_j empty clusters C_{1j}, ..., C_{l_jj} 2 set num_points_{aj} of each C_{aj} ∈ C_j to 1 3 set centroids of each C_{aj} ∈ C to its corresponding init_centroids 4 for x ∈ W_j do 5 ## assign x into the closest centroid argmin_{C_{aj} ∈ C_j} d(x, centroid_{aj}) 6 ## update the total number of points the selected cluster C_{aj} C_{aj}.num_points_{aj} = C_{aj}.num_points_{aj} + 1 7 ## update the centroid of the selected cluster C_a C_{aj}.centroid_{aj} = C_{aj}.centroid_{aj} + $\frac{x - C_{aj}.centroid_{aj}}{C_{aj}.num_points_{aj}}$ 8 end for 9 return C_j </pre>

Algorithm 3: Temporal Neighbor Clustering at S_j.

We now describe how the outlying attribute generator function for data stream j (G_j) forms the local context of an outlier o_i^j detected in W_j and then generates outlying attributes for the outlier. We call the local context of the outlier o_i^j as the inlier class of the outlier.

Fig. 7 depicts how the inlier class is formed for an outlier o_i^j. The inlier class is generated by finding a cluster C_{aj} ∈ C_j whose centroid centroid_{aj} is the closest to the estimated normal values of the outlier (o_i^j). We denote by dist_{C_{aj}}ⁱ the distance between the centroid of cluster C_{aj} (centroid_{aj}) and o_i^j. Recall that the cluster does not keep the data points it has absorbed, but it has the information about the number of data points absorbed by the cluster (num_points_{aj}). Initially, the inlier class only has two members: centroid_{aj} and o_i^j. The additional members are then added by generating auxiliary data points whose distances to centroid_{aj} is less than dist_{C_{aj}}ⁱ. The total number of generated auxiliary data points is equal to num_points_{aj} multiplied by the number of attributes in data stream j (d_j).

The process of generating the auxiliary data points is done by first creating a covariance matrix of size d_j × d_j by multiplying $\frac{dist_{C_{aj}}^i}{3}$ with an identity matrix of size d_j by d_j as follows:

$$\text{Cov}_{inlier} = \frac{dist_{C_{aj}}^i}{3} \cdot \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} = \begin{bmatrix} \frac{dist_{C_{aj}}^i}{3} & 0 & \dots & 0 \\ 0 & \frac{dist_{C_{aj}}^i}{3} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \frac{dist_{C_{aj}}^i}{3} \end{bmatrix}$$

The decision to divide the distance by three is based on the empirical rule that in normally distributed data, nearly all of the data points fall within three standard deviations of the mean [45]. The multivariate normal data generator requires three input parameters: (1) a mean of d_j dimension; (2) a d_j × d_j covariance matrix, and (3) a number of data points to generate. We use centroid_{aj}, Cov_{inlier}, and num_points_{aj} * d_j, as the parameters (1), (2), and (3) for the multivariate normal data generator, respectively.

So far, we have shown the creation of the local context of an outlier o_i^j that we also call the inlier class of o_i^j. The inlier class of o_i^j consists of the estimated normal value of the outlier (o_i^j) the centroid of the closest clusters o_i^j and auxiliary data points. All the members of the inlier class have the same attributes as the data points in stream j. For the purpose of training a classifier that separates the outlier and its local context, a label of value 0 is added to each member of the inlier class.

After generating the inlier class, the outlying attribute generator function for data stream j (G_j) form an outlier class for the outlier o_i^j. Fig. 8 illustrates how the outlier class is formed. The outlier class is created by generating multivariate normal data

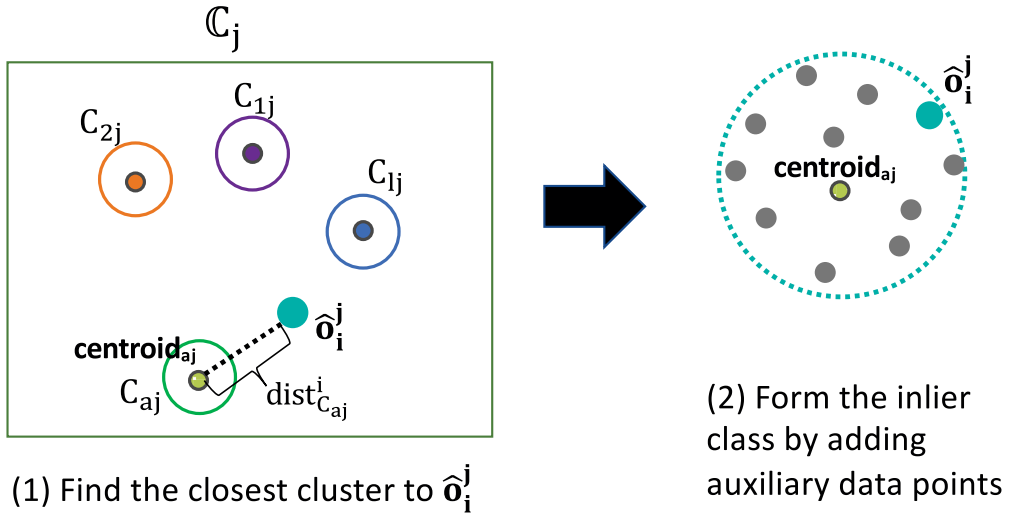


Fig. 7. Forming an inlier class.

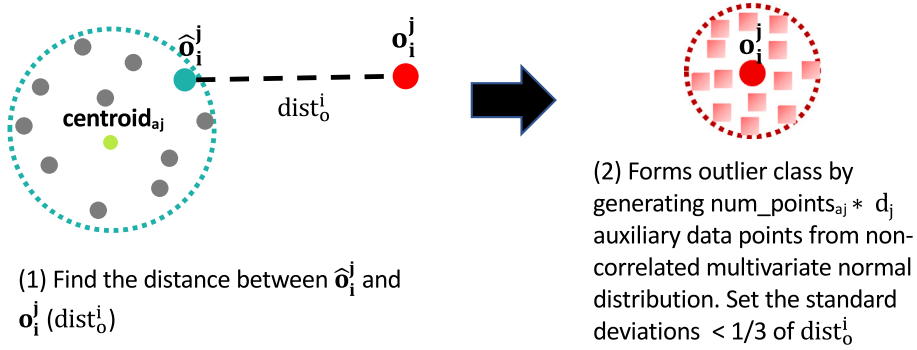


Fig. 8. Forming an outlier class.

centered around $\hat{o}_i^j$. To ensure that the members of the outlier class do not overlap with those of the inlier class, the standard deviation denoted as σ_o is set to be less than one-third of the distance between $\hat{o}_i^j$ and $\hat{o}_i^j$. The $d_j \times d_j$ covariance matrix of the outlier class is generated as follows:

$$\text{Cov}_{\text{outlier}} = \frac{\sigma_o}{3} \cdot \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} = \begin{bmatrix} \frac{\sigma_o}{3} & 0 & \dots & 0 \\ 0 & \frac{\sigma_o}{3} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \frac{\sigma_o}{3} \end{bmatrix}$$

Similar to the inlier class data generation, to generate the outlier class we use $\hat{o}_i^j$, $\text{Cov}_{\text{outlier}}$, and $\text{num_points}_{aj} * d_j$ as the parameters (1), (2), and (3) for the multivariate normal data generator, respectively. The generation of the multivariate data explained in the previous paragraph creates the outlier class of $\hat{o}_i^j$ that consists of $\hat{o}_i^j$ and auxiliary data points. All members of the outlier class have the same attributes as the data points in stream j . Additionally, each member of the outlier class has a label with a value of 1.

After forming the inlier and outlier classes, the subsequent step involves determining a decision boundary or hyperplane that separates both groups (illustrated in Fig. 9) using a linear classifier. Specifically, we train a linear support vector machine (SVM) [46] using data from the inlier and outlier classes. The decision boundary is a linear combination of the data attributes where each attribute corresponds to a coefficient or a weight. The classifier applies lasso regularization [47] such that the resulting hyperplane has zero weights on any attributes that are not relevant in forming the boundary.

Given $D = \{A_{j1}, A_{j2}, \dots, A_{jd_j}\}$ the set of attributes in S_j and d_j the total number of attributes, the hyperplane that separates the inlier class from the outlier class of $\hat{o}_i^j$ is formulated as $w_{j0}^i + w_{j1}^i A_{j1} + w_{j2}^i A_{j2} + \dots + w_{jd_j}^i A_{jd_j} = 0$. The attribute weights

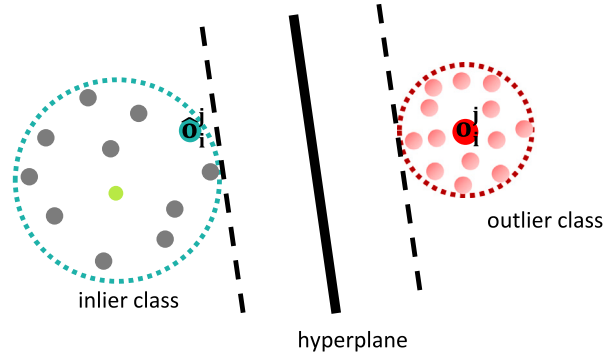


Fig. 9. Finding the decision boundary that separates the inlier class from the outlier class.

Algorithm 4: OutlyingAttrGeneratorAtS_j(O_j, $\hat{O}_j$, C_j, γ, D_j)	
Inputs: O ^j all outliers detected within the active tumbling window (W _j) of data streams j (S _j), $\hat{O}^j$ the estimated normal values of all outliers in W _j , C _j the set of cluster of data points in W _j , γ the threshold of outlying contribution score, and D _j the list of the attributes in S _j	
Output: OutAttrList _j the list of the list of outlying attributes of all outliers within W _j	
1	create an empty list OutAttrList _j
2	for each o _i ^j in O ^j and $\hat{o}_i^j$ in $\hat{O}^j$ do
3	inlier_class = generate_inlier_class(C _j , $\hat{o}_i^j$)
4	outlier_class = generate_outlier_class(o _i ^j , $\hat{o}_i^j$)
5	classifier = LinearClassifier(regularization=l ₁)
6	hyperplane = classifier.train(inlier_class, outlier_class)
7	H _{o_i^j} = absolute(hyperplane.weights)
8	$\text{sum_w}_{o_i^j} = \sum_{b=1}^{d_j} w_{jb}^i $ $f_{o_i^j} = \frac{ H_{o_i^j} }{\text{sum_w}_{o_i^j}}$
9	create an empty list F _{o_i^j}
10	for 1 ≤ b ≤ D _j do :
11	if f _{o_i^j} [b] > γ do :
12	F _{o_i^j} .append(D _j [b])
13	end if
14	end for
15	OutAttrList _j .append(F _{o_i^j})
16	end for
17	return OutAttrList

Algorithm 4: Outlying Attribute Generator at S_j.

associated with $\mathbf{o}_i^j$ on the hyperplane are expressed as a vector $\mathbf{H}_{\mathbf{o}_i^j} \in \mathbb{R}^{d_j}$ where:

$$\mathbf{H}_{\mathbf{o}_i^j} = \begin{bmatrix} w_{j1}^i \\ w_{j2}^i \\ \vdots \\ w_{jd_j}^i \end{bmatrix}$$

Once $\mathbf{H}_{\mathbf{o}_i^j}$ is obtained, the algorithm stores the absolute values of the hyperplane's weights in $|\mathbf{H}_{\mathbf{o}_i^j}| \in \mathbb{R}^{d_j}$ where:

$$|\mathbf{H}_{\mathbf{o}_i^j}| = \begin{bmatrix} |w_{j1}^i| \\ |w_{j2}^i| \\ \vdots \\ |w_{jd_j}^i| \end{bmatrix}$$

$|\mathbf{H}_{\mathbf{o}_i^j}| \in \mathbb{R}^{d_j}$ is then used to calculate the outlier's outlying contribution scores stored in the vector $\mathbf{f}_{\mathbf{o}_i^j} \in \mathbb{R}^{d_j}$ by first computing the sum of all absolute weights in $|\mathbf{H}_{\mathbf{o}_i^j}|$.

$$sum_{w_{\mathbf{o}_i^j}} = \sum_{b=1}^{d_j} |w_{jb}^i|$$

$$\mathbf{f}_{\mathbf{o}_i^j} = \frac{|\mathbf{H}_{\mathbf{o}_i^j}|}{sum_{w_{\mathbf{o}_i^j}}}$$

Each attribute in $\{A_{j1}, A_{j2}, \dots, A_{jd_j}\}$ corresponds to an outlying contribution score in $\mathbf{f}_{\mathbf{o}_i^j}$. The attributes having an outlying contribution score higher than a given threshold γ are determined as the outlying attributes of $\mathbf{o}_i^j$.

Algorithm 4 outlines the process by which $\mathcal{G}_j$ determines the outlying attributes of each outlier within the active window of data stream j . The algorithm initializes an empty list OutAttrList j designated to store the outliers' outlying attributes (Line 1). It then iterates over each outlier (Lines 2–16). Within each iteration, the algorithm forms the inlier and outlier classes (Lines 3–4) and then uses data from those classes to determine a hyperplane that separates the inlier class from the outlier class (Lines 5–6). The absolute values of the attributes' weights on the hyperplane are then used to compute the outlying contribution scores of the outlier's attributes (Lines 7–8). The identification of the outlying attributes involves comparing each attribute's outlying contribution score with a predetermined threshold. If its outlying contribution score exceeds the threshold, the corresponding attribute is added to the list of outlying attributes of $\mathbf{o}_i^j$ ($F_{\mathbf{o}_i^j}$) (Lines 9–15). Finally, the algorithm returns OutAttrList j , which is a list of $|\mathbf{O}^j|$ outlying attribute lists, where $|\mathbf{O}^j|$ corresponds to the number of outliers within W_j , the active window of data stream j .

6. Complexity analysis of EXOS

In this section, we discuss the time and space complexity of EXOS based on the following parameters:

- N_j , the number of data points in the active tumbling window of data stream j (W_j), where $N_j = \frac{\text{tw}_s}{\text{arrivalrate}_j}$, tw_s is a constant representing the tumbling window size, and arrivalrate_j is the data arrival rate at stream j . We also assume that N_j is the number of the data points representing each data stream during the initialization phase.
- $|\mathbf{O}^j|$ the number of outliers in W_j ,
- D , the total number of attributes of all data streams. $D = \sum_{j=1}^m d_j$ where d_j is the number of attributes in data stream j (S_j)
- k , the number of eigenvectors
- m , the number of data streams
- l_j the number of clusters in S_j , the number of clusters in each stream can vary
- I , the maximum number of iterations required to initialize clusters of each stream
- n_{samples} , the maximum number of auxiliary data points generated for the inlier or outlier classes.
- epochs , the number of iterations to train a linear SVM

6.1. Time complexity of EXOS

We now discuss the time complexity of EXOS' offline and online phases.

6.1.1. Time complexity of the offline phase

The initialization or offline phase of EXOS involves two tasks: (1) initializing k eigenvectors $\mathbf{Q}$ (Algorithm 1, Line 1) and (2) initializing l_j number of clusters in each data stream (Algorithm 1, Line 2). Task 1 uses N_j combined data points of D attributes as input to the naive PCA algorithm [96]. Naive PCA requires $O(N_j D^2)$ time complexity to compute a covariance matrix of $D \times D$ size, $O(D^3)$ for eigen decomposition of the covariance matrix, and $O(D \log D)$ to sort k eigenvalues and select k eigenvectors to form $\mathbf{Q}$. Overall the time complexity for Task 1 is $O(N_j D^2 + D^3 + D \log D)$.

Task 2 can be done in parallel since creating l_j initial clusters for a data stream does not depend on another stream. The cluster initialization for S_j is done using the k-means clustering algorithm that requires $O(N_j l_j I d_j)$ time complexity [48].

The overall time complexity of the Initialization Phase is $O(N_j D^2 + D^3 + D \log D) + O(N_j l_j I d_j) = O(N_j D^2 + D^3 + D \log D + N_j l_j I d_j)$. Assuming that the number of data points N_j is larger than the total number of attributes D , we can summarize the time complexity to $O(N_j D^2 + N_j l_j I d_j)$.

6.1.2. Time complexity of the online phase

The complexity of the EXOS' online phase is divided into three parts (1) Estimator, (2) Temporal Neighbor Clustering, and (3) Outlying Attribute Generator.

Time complexity of estimator. We compute the time complexity of the Estimator, Est, based on the active windows of all data streams. Est computes the number of timestamp steps needed to align data points from m streams (Algorithm 2, Line 1). This process has $O(m)$ time complexity and is followed by $O(m N_j)$ to align and select data points (Algorithm 2, Line 2). Notice that the number of selected aligned data points is equal to N_j when the arrival rates of all streams are equal, otherwise, it is less than N_j .

Est takes $O(1)$ to get the total number of data points in AlignedData (Algorithm 2, Line 3). It takes constant complexity because AlignedData is a key-value set or hashmap data structure and getting the total number of data points is the same as getting the size of AlignedData. Recall that in a typical hashmap implementation, the size (the number of key-value pairs) is tracked directly as an attribute of the hashmap. This means the size attribute requires no iteration or complex computation, just a simple read or lookup at the stored value.

Est initializes a zero matrix $\mathbf{Z}$ of size $D \times k$ which takes $O(Dk)$ time complexity (Algorithm 2 Line 4). It goes through every data point $x \in \mathbb{R}^D$ in the selected aligned data and updates $\mathbf{Z} = \mathbf{Z} + \frac{1}{N_j} \mathbf{x}_{\text{aligned}} \mathbf{x}_{\text{aligned}}^T \mathbf{Q}$. Multiplying $\mathbf{x}_{\text{aligned}} \mathbf{x}_{\text{aligned}}^T$ results in a matrix $D \times D$ and the time complexity is $O(D^2)$. Multiplying the resulting matrix with $\mathbf{Q}$ takes $O(D^2 k)$ and adding the result to $\mathbf{Z}$ takes $O(Dk)$. Overall updating $\mathbf{Z}$ for N_j times has $O(N_j (D^2 + D^2 k + Dk))$ time complexity (Algorithm 2, Lines 5–8). After updating $\mathbf{Z}$, the next step is to update $\mathbf{Q}$ using QR Decomposition of $\mathbf{Z}$ (Algorithm 2, Line 9). The standard QR Decomposition uses the Householder Transformation algorithm [49]. It involves a series of reflections to transform each column in $\mathbf{Z}$ into an upper triangular form. The reflection involves vector and matrix multiplication operations that use $O(D^2)$. Since there are k columns in $\mathbf{Z}$, the overall time complexity for QR decomposition is $O(D^2 k)$.

Copying the updated $\mathbf{Q}$ takes $O(Dk)$ and dividing the copied $\mathbf{Q}$ into m segments and storing them in a list of length m requires $O(m)$ operations. Hence the time complexity for the function slice_eigenvectors is $O(Dk + m)$ (Algorithm 2, Line 10).

It takes $O(m)$ to initialize est_list (Algorithm 2, Line 11). In parallel, each slice of $\mathbf{Q}$, denoted as $\mathbf{Q}_j$, is used to estimate the outliers detected in W_j . The operation $\hat{\mathbf{O}}^j = \mathbf{O}^j \mathbf{Q}_j \mathbf{Q}_j^T$ requires $O(d_j k)$ to transpose $\mathbf{Q}_j$, $O(|\mathbf{O}^j| d_j k)$ for the $\mathbf{O}^j \mathbf{Q}_j$ multiplication and $O(|\mathbf{O}^j| k d_j)$ to multiply the intermediate result with $\mathbf{Q}_j^T$. Overall the estimation process takes $O(d_j k + 2(|\mathbf{O}^j| d_j k))$ time complexity for each data stream (Algorithm 2, Line 12–17).

In summary, Est requires: $O(1 + 3m + m N_j + 2Dk + D^2 k + N_j Dk + N_j D^2 + N_j D^2 k + d_j k + 2(|\mathbf{O}^j| d_j k))$ to process all its tasks for all data streams. Since $N_j \gg D$, $D > k$, $N_j \gg |\mathbf{O}^j|$ and $D \gg d_j$, the dominant term here is $O(N_j D^2 k)$, thus the time complexity for Est is $O(N_j D^2 k)$.

Time complexity of temporal neighbor clustering. The Temporal Neighbor Clustering component runs m Cf_j functions in parallel. Each Cf_j is associated with a data stream j . It takes $O(l_j)$ to set l_j empty clusters, $O(l_j)$ to initialize num_points_{aj} , the number of data points of every cluster, and $O(l_j d_j)$ to set centroid_{aj} of each cluster (Algorithm 3, Lines 1–3). For each data point in the active tumbling window of data stream j (W_j), Cf_j requires $O(l_j d_j)$ to assign the data point into a cluster, $O(1)$ to update the assigned cluster's num_points_{aj} , and $O(d_j)$ to update its centroid_{aj} . Considering there are N_j data points in the W_j , the time complexity for updating the clusters is $O(N_j (l_j d_j + d_j + 1))$ (Algorithm 3, Lines 4–7). In summary, the operations in Cf_j have a time complexity of $O(2l_j + d_j + N_j (l_j d_j + d_j + 1))$, where the dominant term is $O(N_j l_j d_j)$, resulting in time complexity $O(N_j l_j d_j)$ for Cf_j .

Time complexity of outlying attribute generator. The outlying attribute generator component executes m G_j functions simultaneously. Each G_j takes $O(1)$ to initialize an empty list OutAttrList $_j$. It identifies the outlying attributes of each $\mathbf{o}_i^j$ in $\mathbf{O}^j$ by first generating n_{samples} auxiliary data points to form an inlier class for $\mathbf{o}_i^j$ based on the $\hat{\mathbf{o}}_i^j$. The process of forming the inlier class involves $O(l_j)$ time to find the closest cluster to $\hat{\mathbf{o}}_i^j$, $O(d_j^2)$ time to generate a covariance matrix of size $d_j \times d_j$, and $O(n_{\text{samples}} d_j^2)$ time to generate n_{samples} multivariate normal data [50]. Consequently, forming the inlier class requires a total of $O(l_j + d_j^2 + n_{\text{samples}} d_j^2)$. It takes the same time complexity to form the outlier class.

After forming the inlier and outlier classes, the next step is to determine the hyperplane that separates both classes using Linear SVM. Both inlier and outlier classes have n_{samples} data points; thus, the total number of training data is $2n_{\text{samples}}$. The process of training a linear SVM using Stochastic Gradient Descent (SGD) [51] takes $O(\text{nepochs } 2n_{\text{samples}} d_j)$, where nepochs , $2n_{\text{samples}}$, and d_j represent the number of iterations in SGD, the total number of training samples for the linear SVM, and the number of attributes

in data stream j , respectively. After the hyperplane is identified, G_j computes the absolute values of the hyperplane weights in $O(d_j)$ time. It then takes $O(d_j)$ to compute the outlying contribution scores of the attributes, $O(d_j)$ to check whether the outlying contribution score of each attribute exceeds the threshold for outlying attributes, and $O(d_j)$ to store the outlying attributes of σ_i^j .

In total to generate the outlying attributes of an outlier σ_i^j , $O(l_j + 4d_j + d_j^2 + n_{\text{samples}}d_j^2 + \text{nepochs } 2n_{\text{samples}}d_j)$. Considering the dominating term, the time complexity to generate the outlying attributes of an outlier σ_i^j is $O(\text{nepochs } 2n_{\text{samples}}d_j)$. Thus, for all outliers in W_j , G_j requires $O(|\mathbf{O}^j| \text{nepochs } 2n_{\text{samples}}d_j)$ time complexity.

The overall time complexity of the online phase. EXOS takes $O(mN_jD)$ to absorb data points within the active tumbling windows of all data streams (Algorithm 1, Line 6). Additionally, it needs $O(|\mathbf{O}^j|d_j)$ time to get outliers from each stream in parallel (Algorithm 1, Line 7). As the estimator and temporal neighbor clustering components run in parallel (Algorithm 1, Lines 8–13), their time complexity is $O(\max(N_jD^2k, N_jl_jd_j))$. The outputs of these components serve as the inputs to the outlying attributes generator component which operates m parallel functions (Algorithm 1, Lines 14–19), each demanding $O(|\mathbf{O}^j| \text{nepochs } 2n_{\text{samples}}d_j)$. The overall time complexity of the EXOS's Online Phase sums up to $O(mN_jD + |\mathbf{O}^j|d_j + \max(N_jD^2k, N_jl_jd_j) + |\mathbf{O}^j| \text{nepochs } 2n_{\text{samples}}d_j)$.

Given that $N_j \gg D$, $N_j \gg |\mathbf{O}^j|$, $N_j \gg l_j$, $n_{\text{samples}} > d_j$, and $D > d_j$, we can simplify the overall time complexity of the Online Phase to $O(N_jD^2k + |\mathbf{O}^j| \text{nepochs } 2n_{\text{samples}}d_j)$.

6.2. Space complexity of EXOS

We now discuss the memory space complexity of EXOS' initialization and Online Phases. When dealing with data streams, it is important to keep relevant data structures in the memory. Accessing data from memory is almost instantaneous, whereas I/O operations involve reading data from or writing data to disk, which is significantly slower. By performing operations in the memory, the algorithm can process data more quickly, which is crucial for real-time applications.

6.2.1. Space complexity of the initialization phase

To get the initial eigenvector matrix $\mathbf{Q}$ (Algorithm 1, Line 1), EXOS applies the Naive PCA algorithm. This algorithm employs $D \times D$ matrix for correlation matrix derived from the initial data, where D represents the total number of attributes in all data streams. The algorithm then decomposes the correlation matrix using eigen decomposition into a list of eigenvalues of size D and a matrix of eigenvectors of size $D \times D$. Thus, to construct the initial eigenvectors $\mathbf{Q}$, EXOS requires $O(D^2)$ space to store the correlation matrix, $O(D^2)$ to store the eigenvectors, and $O(D)$ to store the eigenvalues. Taking the maximum term, the space complexity required by Naive PCA is $O(D^2)$. Note that in Algorithm 1, Line 1, the function that calls Naive PCA only returns the first k eigenvectors. Thus $\mathbf{Q}$ is a $D \times k$ matrix that requires $O(D \times k)$ space.

EXOS stores a list of m sets of clusters of m data streams denoted as AllClust (Algorithm 1, Line 2). To initialize AllClust, EXOS runs the k -means clustering algorithm on each data stream j using N_j initial data points. Each data point has d_j dimension. The k -means algorithm groups data points into l_j clusters. It needs $O(N_jd_j)$ space to store data points in the clusters, $O(N_jl_j)$ space to store the distance between all data points to all clusters, $O(l_jd_j)$ space to store centroids. Hence, the space complexity for k -means in all data streams is $O(m(N_jd_j + N_jl_j + l_jd_j))$. Recall that in Algorithm 1, Line 2, the function that employs the k -means algorithm on each data stream only return the centroids of the constructed clusters. Thus, the AllClust itself only uses $O(ml_jd_j)$ space. The total space complexity of the Initialization Phase is $O(D^2 + m(l_jd_j + N_j + N_jd_j))$.

6.2.2. Space complexity of the online phase

During the Online Phase (Algorithm 1 Lines 5–22), since there are m data streams, EXOS maintains m buffers or tumbling windows denoted as TWInS which store data within a period of time and replace the data when their period expires (Algorithm 1, Line 6). Each buffer denoted as W_j corresponds to a data stream j and requires approximately $O(N_jd_j)$ space. Recall that d_j is the number of attributes in stream j and N_j is the number of data points in W_j which is calculated as follow $N_j = \frac{\text{tw}_s}{\text{arrivalrate}_j}$, where tw_s is a constant representing the tumbling window size and arrivalrate_j is the data arrival rate at stream j .

Throughout the Online Phase, EXOS uses $O(m|\mathbf{O}^j|(d_j + 1))$ space to store all outliers detected within m tumbling windows of all streams (Algorithm 1, Line 7). Recall that $|\mathbf{O}^j|$ is the number of outliers in all data streams, and each outlier in a stream j has d_j attributes and a timestamp associated with it. In addition to the space needed to store the outliers, $O(m|\mathbf{O}^j|(d_j))$ space is also needed to keep the corresponding estimated normal values of the outliers in the est_list data structure (Algorithm 1, Line 9). Furthermore, EXOS maintains $O(Dk)$ space for $D \times k$ eigenvectors $\mathbf{Q}$ updated in the memory continuously (Algorithm 1, Line 9). EXOS maintains AllClust which stores $O(ml_jd_j)$ centroids.

The space complexity of each key component of EXOS is further discussed in the following sections.

Space complexity of estimator. The space complexity analysis for Est involves considering various operations. Est takes $O(1)$ space for to keep the number of time steps ($n_{\text{ts_step}}$) needed to align data points from m tumbling windows (Algorithm 2, Line 1). It uses up to $O(mN_jD)$ space to store a key-value set of the aligned data, AlignedData (Algorithm 2 Line 2), and $O(1)$ for variable $\bar{N}$ that represents the total number of aligned data points (Algorithm 2 Line 3).

Furthermore, EXOS utilizes $O(Dk)$ space for initializing $D \times k$ zeros matrix $\mathbf{Z}$ (Algorithm 2 Line 4). When iterating over each data point to update $\mathbf{Z}$, it requires $O(D)$ space for $\mathbf{x}_{\text{aligned}} \in \mathbb{R}^D$, $O(D^2)$ space to keep the intermediate result of $\mathbf{x}_{\text{aligned}}\mathbf{x}_{\text{aligned}}^T$ operation, and $O(Dk)$ space for multiplying the intermediate result with the eigenvectors $\mathbf{Q}$ and adding the result to $\mathbf{Z}$ (Algorithm 2 Lines 5–8).

The Q-R decomposition can be implemented by modifying $\mathbf{Z}$ in place. The space complexity for Algorithm 2 Line 9 is $O(1)$ since it does not require an additional memory proportional to the input size. The process of slicing the eigenvectors $\mathbf{Q}$ (Algorithm 2 Line 10) requires $O(Dk)$ space to duplicate $\mathbf{Q}$ and store it into m parts. The parallel procedure to estimate the normal values of outliers (Algorithm 2 Lines 12–17) requires approximately $O(mN_j(k + D))$ space.

Summing up all the space complexities of Est's operations, we get $O(1 + mN_jD + 3Dk + D^2 + mN_j(k + D))$. Considering $N_j \gg D$, the upper bound on the space required by Est is $O(mN_j(k + D))$ as it is the most significant term in the sum.

Space complexity of temporal neighbor clustering. We now discuss the space complexity analysis for the Temporal Neighbor Clustering Component based on the operation needed by its m independent functions $Cf_1, Cf_2, \dots, Cf_m$. Each Cf_j is associated with a data stream j that has d_j attributes and l_j clusters. A Cf_j takes $O(l_jd_j)$ space to set l_j initial centroids of all clusters (each $\text{centroid}_{ij} \in \mathbb{R}^{d_j}$) and $O(l_j)$ space for all variables that keep the number of data points absorbed by the clusters (Algorithm 3, Line 1–2). Additionally, when iterating over each data point x to assign it to a cluster, Cf_j requires $O(l_jd_j)$ space when computing the distance of x to l_j clusters and $O(d_j)$ space for updating the centroid of the closest cluster to x (Algorithm 3, Line 4–8).

In total, a Cf_j uses $O(l_j + 2l_jd_j + d_j)$. Thus, the overall space required by the Temporal Neighbor Clustering component is $O(m(l_j + 2l_jd_j + d_j))$. Taking the most dominant term, the space complexity of the Temporal Neighbor Clustering component is $O(m(2l_jd_j))$.

Space complexity of outlying attribute generator. To analyze the space complexity of the Outlying Attribute Generator component, we need to examine the space complexity of m independent functions $G_1, G_2, \dots, G_m$. Each G_j corresponds to a data stream j .

Recall that in each active tumbling window associated with a stream j , the total number of outliers detected is denoted as $|\mathbf{O}|$. In a G_j , each outlier σ_i^j has an inlier class and an outlier class. Each class has n_{samples} auxiliary data points of $d_j + 1$ attributes (the original attributes of the outlier + the label of the data points). Thus the G_j function requires $O(n_{\text{samples}}(d_j + 1))$ space to store the generated inlier class (Algorithm 4, Line 3) and $O(n_{\text{samples}}(d_j + 1))$ space for an outlier class (Algorithm 4, Line 3).

During the training of a linear classifier (Algorithm 4, Line 5) to find the decision boundary or hyperplane that separates the inlier and outlier classes, the classifier algorithm requires $O(2n_{\text{samples}}(d_j + 1))$ space to copy and store the entire data points from both the inlier and outlier classes, $O(d_j)$ to keep the weights associated with each attribute, and $O(1)$ space for the bias term. These are general space requirements for the linear classifier. Depending on the implementation of the classification algorithm, additional space might be needed for other parameters such as learning rate, etc [52].

The hyperplane generated by the linear classifier for each outlier σ_i^j is kept by G_j in a vector of size d_j ; thus, it takes $O(d_j)$ space for the hyperplane (Algorithm 4, Line 6). Additionally, G_j needs $O(d_j)$ space to keep the absolute values of attribute weights of the hyperplane (Algorithm 4, Line 7). It also uses $O(d_j)$ space to store the outlying contribution scores of the attributes in a vector $\mathbf{f}_{\sigma_i^j}$ of size d_j (Algorithm 4, Line 8). Finally, to store the outlying attributes of σ_i^j , G_j uses $O(d_j)$ space for a list named $F_{\sigma_i^j}$ (Algorithm 4, Line 9–14).

In summary, the upper bound of G_j 's space complexity for each outlier σ_i^j is $O(2n_{\text{samples}}(d_j + 1))$ because this is the most dominant term. Thus, to generate outlying attributes of all $|\mathbf{O}|$ outliers detected in an active tumbling window of stream j , G_j requires $O(|\mathbf{O}|2n_{\text{samples}}(d_j + 1))$ space. We can conclude that the Outlying Attribute Generator's space complexity is $O(m|\mathbf{O}|2n_{\text{samples}}(d_j + 1))$.

7. Performance evaluation

We conduct comprehensive experiments to evaluate the performance of EXOS. In this section, we first discuss our experimental setup and then provide our experimental results and analysis.

7.1. Experimental setup

The experimental setup consists of (i) competing algorithms, (ii) datasets, (iii) performance metrics, and (iv) software and hardware.

7.1.1. Competing algorithms

We compare EXOS to four existing algorithms reviewed in Section 2: COIN [28], MICOES [33], MacroBase [31,41], and EXstream [30,41]. Like EXOS, all these algorithms are agnostic to the algorithm used for outlier detection. COIN is one of the algorithms that use inlier neighbors of an outlier as a local context to find the outlier's outlying attributes. It is designed for static data, yet we run it in batches to compare it with EXOS. The other three algorithms are proposed for data streams. EXstream is intended for interval or collective outliers, yet it can also be applied for point outliers. The source code of each algorithm is publicly available. In Section 2, we review four algorithms proposed for finding outliers's outlying attributes in the data stream. However, we exclude EXAD since the published work does not provide a detailed explanation and implementation of the algorithm.

Table 3
Summary of the datasets.

Datasets	# Streams	Total number of data points	# Outliers injected	# Attributes per stream
Microtremor	4	7M	70K (1%)	3
AMPds2	25	26M	260K (1%)	2, 3, 11
Intel	9	401K	4.5K (1.12%)	4
Synthetic 1	5	50K	500 (1%)	10
Synthetic 2	4	40K	500 (1%)	10

7.1.2. Datasets

We use three real-world datasets that reflect multi-source data streams: (1) Intel, (2) Microtremor, and (3) AMPds2. The Intel dataset [53] was gathered from 54 sensors deployed in the Intel Berkeley Research Lab between February 28th and April 5th, 2004. Each sensor sent time-stamped data with attributes: humidity, temperature, light, and voltages every 31 s. Since the sensors started sending measurements on different dates, we simulated data gathered between March 1st–31st, 2004 from Sensors 7, 9, 23, 25, 26, 29, 36, 38, and 44.

The Microtremor dataset [54] was collected from 33 seismic recording stations in Sacramento–San Joaquin Delta Region California. The ambient microtremor data were collected using temporary broadband seismometers over continuous time intervals ranging between 2 and 2.8 h. After examining the metadata, we decide to use the data collected from 4 stations with site IDs, 2030, 2031, 2032, and 2033, as their data from these sites were collected on the same day while the data from the other sites were collected on different days.

The AMPds dataset [55] contains electricity, water, and natural gas measurements at one-minute intervals in a residential house in Canada. There are 21 power meters, 3 water meters, and 2 natural gas meters. Each meter sent a total of 1,051,200 readings for 2 years (April 2012 to March 2014). Power, water, and gas have 11, 2, and 3 attributes, respectively.

These datasets do not have the ground truth information for outliers and their outlying attributes. For performance evaluation purposes, we synthetically generated that information. Specifically, we injected outliers into the dataset by randomly selecting around 1% of the original data points to be outliers. For each selected outlier, we randomly selected several attributes from the dataset to be the outlying attributes and set the value of each of those attributes to be far away from its mean value.

For example, suppose in the Intel dataset, data point #100 is chosen as an outlier and temperature as the outlying attribute. The temperature value of data point #100 is then set to either $\max(\text{temperature}) + \text{delta} * \text{standarddeviation}(\text{temperature})$ or $\min(\text{temperature}) - \text{delta} * \text{standarddeviation}(\text{temperature})$, where $\text{delta} > 3$. The summary of the datasets used for the evaluation is provided in Table 3.

In addition, we create synthetic datasets that consist of five and four groups of data whose attributes are correlated. Each group represents a data stream and has 10 attributes. To ensure that the data streams are correlated, we first generate a correlation matrix using a technique described in [56]. This technique allows us to specify whether the data attributes within and across data streams are highly or weakly correlated. The correlation matrix is then used to generate multivariate normal data. We inject outliers into the dataset in the same way as we do for the real datasets.

7.1.3. Performance metrics

We measure the effectiveness of the algorithms in finding outlying attributes using average precision, average recall, and average F1 score. For each outlier, we compute the numbers of True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN) of its outlying attributes. We then use the information to compute:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1 Score} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

For example, suppose tuple #5 in the Intel dataset is an outlier with ground truth outlying attributes = light. If EXOS decides its outlying attributes = light, voltages, the data point #5's confusion matrix is as follows:

d = 4	Predicted: No	Predicted: Yes
Ground Truth: No	TN = 2	TP = 1
Ground Truth: Yes	FN = 0	FP = 1

Given the confusion matrix, we can compute the outlying attribute's precision, recall, and F1 score of the outlier. Precision = $\frac{1}{1+1} = 0.5$, recall = $\frac{1}{2+0} = 0.5$, F1 score = $\frac{2*0.5*0.5}{0.5+0.5} = 0.5$. We compute the precision, recall, and F1 score of each outlier and then aggregate the average of each metric.

In addition to measuring the accuracy of the algorithms, we also consider their efficiency. Given that we are dealing with a continuous arrival of data, we need to ensure that the algorithms can keep up with the incoming data promptly. Therefore, we also measure the average explanation time the algorithms take to identify the outlying attributes of an outlier, which we refer to as the average explanation time, as part of our evaluation process.

7.1.4. Software & hardware

We implement EXOS in Python 3 and simulate the multi-source data streams in parallel using the multiprocessing package. Our source code is available on GitHub <https://github.com/egawati/exos>. We use the Python implementations of MacroBase and EXstream that are provided for the Exathlon benchmark [41]. We add some helper functions to the competitive algorithms so that they can run in parallel as we simulate multiple data streams. The experiments were conducted on a MacBook Pro: macOS Catalina, Processor 2.7 GHz Quad-Core Intel Core i7, Memory 16 GB 1600 MHz DDR3.

7.2. Performance results and analysis

In this section, we report the overall performance of the algorithms on each dataset and study the impact of window size, data arrival rate, and concept drift on the performance of each algorithm. Finally, we investigate how some parameters of EXOS affect its performance.

7.2.1. Overall performance

Concurrent data streams. We evaluate the algorithms by simulating each dataset described in Section 7.1.2 as concurrent data streams where all data streams have the same arrival rate. Each algorithm has access to the tumbling windows. In each data stream, we set the window size to accommodate 1440 data points (1 day) for Intel and AMPds2 and 1000 data points for Microtremor and Synthetic.

Table 4 shows that EXOS has the highest average precision for Intel and AMPds2. EXstream gets the best average precision for Microtremor and Synthetic 2 while COIN wins on Synthetic 1. When it comes to average recall, MacroBase wins for AMPds2, Synthetic 1 and 2, while COIN dominates for Intel. For Microtremor, the four algorithms achieve an average recall of 0.99, while EXstream 0.87. Obviously, there is a tradeoff between precision and recall among these algorithms. Precision measures the extent of error by False Positives, while recall deals with the error caused by False Negatives. F1 score balances those scores. EXOS achieves the best F1 score in all the datasets except for AMPds2. The F1 scores of EXOS are 22%, 40%, 4%, 18%, and 37% better than those of COIN, 24%, 49%, 10%, 28%, and 40% better than those of MICOES, 26%, 38%, -6%, 38%, 41% better than those of MacroBase, and 19%, 2%, 6%, 26%, and 33% better than those of EXstream for the Intel, Microtremor, AMPds2, Synthetic 1, and Synthetic 2 datasets, respectively.

Compared to COIN, MICOES, Macrobase, and EXstream which operate independently on each data stream, EXOS stands out as the sole algorithm that incorporates the cross-correlation among all data streams, leveraging it to estimate the normal values of outliers. This unique approach elucidates why EXOS achieves a balanced average precision and recall, distinguishing it from other algorithms that may demonstrate high average precision but lower average recalls. Additionally, although COIN and MICOES similarly utilize the weights of a linear classifier's hyperplane to determine outlying attributes of an outlier, their average F1 score is comparatively lower than that of EXOS. This emphasizes the effectiveness of EXOS's strategy in considering the cross-correlation among data streams.

Table 4 shows that for all the datasets, EXOS has the fastest average execution time for generating outlying attributes for outliers. EXOS is 4.2, 15, 6.4, 1.5, and 1.3 times faster than COIN, 2, 29.6, 2.8, 1.1, and 4 times faster than MICOES, and 1.6, 2.3, 13.8, 12.7, and 10.8 times faster than MacroBase for the Intel, Microtremor, AMPds2, Synthetic 1, and Synthetic 2 datasets, respectively. Even though EXOS and EXstream have similar average execution times, EXOS outperforms EXstream in the average F1 scores for all the datasets. These results underscore that while considering data streams cross-correlation introduces added complexity to EXOS, executing the Estimator component along m functions in the Temporal Neighbor Clustering component in parallel helps speed up the average outlier explanation time on each window.

We also delve deeper into how the means of the average F1 score and explanation time differ among the algorithms. Employing a One-way ANOVA [57] followed by a pairwise Tukey's Honest Significant Difference (HDS) test [58], we aim to discern statistical differences in the performance means of the algorithms. The null hypothesis of the One-way ANOVA posits no significant difference between the means of the algorithms' performance. We ran experiments ten times on each dataset and recorded the results. The One-Way ANOVA test on the average F1 score yields a remarkably small p -value (less than 0.05), providing grounds to reject the null hypothesis. With the understanding that there are statistically significant differences between the means of the algorithms' average F1 scores, we proceed to Tukey's HDS to pinpoint the significant differences between their means.

The Tukey's HDS results are presented in Fig. 10, where the mean difference (meandiff) is calculated by subtracting the mean of group 1 from group 2 (group 2 - group 1). The p -adj column signifies the adjusted p -value, indicating the probability of observing a result as equal to, or more extreme than, the one obtained if the null hypothesis is true. The reported adjusted p -value of the Tukey HSD test in this paper is generated using the Python library `statsmodels.stats.multicomp`. When the adjusted p -value is too small for the software to display, it is shown as 0.0 instead of 0.0000.

A smaller p -adj suggests stronger evidence against the null hypothesis. In this context, the null hypothesis is 'there is no significant difference between the performance means of the algorithms.' The lower and upper columns denote the confidence intervals for the differences between the performance means of the compared algorithms. If the confidence interval excludes zero, it lends support to the rejection of the null hypothesis. The reject column denotes whether the null hypothesis is rejected (True) or not (False).

In Fig. 10(a), the mean difference of the average F1 score between COIN (group 1) and EXOS (group 2) is 0.242, indicating that EXOS's mean is higher than COIN's. The small, adjusted p -value (-0.0) and the mean difference confidence interval excluding zero lead to the rejection of the null hypothesis, signifying a significant difference in the mean of COIN and EXOS average F1 scores. Conversely, there are three comparisons where there is insufficient evidence to reject the null hypothesis: COIN vs Macrobase, COIN

Table 4
Algorithm performance evaluation on multisource data stream.

Datasets	Algorithms	Avg precision	Avg recall	Avg F1 score	Avg execution time (second)
Intel	COIN	0.49	0.99	0.63	0.042
	MICOES	0.5	0.91	0.61	0.020
	MacroBase	0.49	0.92	0.59	0.016
	EXstream	0.85	0.59	0.66	0.011
	EXOS	0.95	0.84	0.85	0.010
Microtremor	COIN	0.33	0.99	0.49	0.045
	MICOES	0.33	0.99	0.50	0.089
	MacroBase	0.34	0.99	0.51	0.007
	Exstream	0.87	0.87	0.87	0.003
	EXOS	0.83	0.99	0.89	0.003
AMPds2	COIN	0.43	0.86	0.54	0.045
	MICOES	0.39	0.80	0.48	0.020
	MacroBase	0.56	0.89	0.64	0.097
	Exstream	0.91	0.42	0.52	0.007
	EXOS	0.94	0.48	0.58	0.007
Synthetic 1	COIN	1.00	0.52	0.64	0.072
	MICOES	0.47	0.79	0.54	0.055
	MacroBase	0.3	1.00	0.44	0.599
	Exstream	0.98	0.45	0.56	0.050
	EXOS	0.84	0.87	0.82	0.047
Synthetic 2	COIN	0.42	0.86	0.54	0.076
	MICOES	0.51	0.58	0.51	0.234
	MacroBase	0.34	1.00	0.50	0.629
	Exstream	0.99	0.44	0.58	0.093
	EXOS	0.88	0.97	0.91	0.058

(a) Tukey's HDS on the algorithms average F1 Score

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	0.242	-0.0	0.1923	0.2917	True
coin	exstream	0.07	0.0013	0.0203	0.1197	True
coin	macrobase	-0.032	0.3945	-0.0817	0.0177	False
coin	micoes	-0.04	0.1792	-0.0897	0.0097	False
exos	exstream	-0.172	-0.0	-0.2217	-0.1223	True
exos	macrobase	-0.274	-0.0	-0.3237	-0.2243	True
exos	micoes	-0.282	-0.0	-0.3317	-0.2323	True
exstream	macrobase	-0.102	0.0	-0.1517	-0.0523	True
exstream	micoes	-0.11	0.0	-0.1597	-0.0603	True
macrobase	micoes	-0.008	0.992	-0.0577	0.0417	False

(b) Tukey's HDS on the algorithms average explanation time

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	-0.031	0.7774	-0.1048	0.0428	False
coin	exstream	-0.0232	0.9098	-0.097	0.0506	False
coin	macrobase	0.2136	0.0	0.1398	0.2874	True
coin	micoes	0.0276	0.8425	-0.0462	0.1014	False
exos	exstream	0.0078	0.9984	-0.066	0.0816	False
exos	macrobase	0.2446	-0.0	0.1708	0.3184	True
exos	micoes	0.0586	0.1902	-0.0152	0.1324	False
exstream	macrobase	0.2368	-0.0	0.163	0.3106	True
exstream	micoes	0.0508	0.3248	-0.023	0.1246	False
macrobase	micoes	-0.186	0.0	-0.2598	-0.1122	True

Fig. 10. The Tukey's HDS test results on the algorithms' overall performance.

Table 5
The algorithms' performance on Synthetic 2 simulated as non-concurrent data streams.

Algorithm	Avg precision	Avg recall	Avg F1 score	Avg explanation time (second)
COIN	0.350	0.683	0.435	0.411
MICOES	0.354	0.727	0.443	0.332
Macrobase	0.258	1.000	0.391	1.994
EXstream	0.988	0.517	0.643	0.284
EXOS	0.933	0.987	0.947	0.331

vs MICOES, and Macrobase vs MICOES. This implies that COIN, Macrobase, and MICOES exhibit similar average F1 scores. Notably, when comparing EXOS (group 1) to EXstream, Macrobase, or MICOES (group 2), the negative meandiff value indicates that EXOS's average F1 score means are greater than that of those three algorithms.

The null hypothesis for Fig. 10(b) is "that there is no significant mean difference in the average explanation time of the algorithms". Four comparisons COIN vs Macrobase, EXOS vs Macrobase, Exstream vs Macrobase, and Macrobase vs MICOES plausibly reject the null hypotheses. The outcomes confirm that only Macrobase has a higher average explanation time, while the other algorithms demonstrate comparable average explanation time.

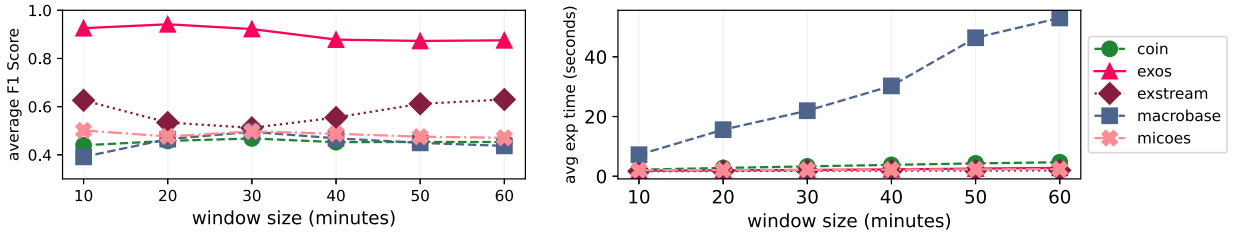


Fig. 11. The impact of the window size on the performance of the algorithms.

Varying window size , fixed arrival rate

(a) Tukey's HDS on the algorithms average F1 Score

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	0.4487	0.0	0.395	0.5024	True
coin	exstream	0.124	0.0	0.0703	0.1777	True
coin	macrobase	-0.0029	0.9999	-0.0566	0.0508	False
coin	micoes	0.0307	0.4652	-0.023	0.0844	False
exos	exstream	-0.3247	0.0	-0.3784	-0.271	True
exos	macrobase	-0.4516	0.0	-0.5053	-0.3979	True
exos	micoes	-0.4181	0.0	-0.4718	-0.3643	True
exstream	macrobase	-0.1269	0.0	-0.1806	-0.0732	True
exstream	micoes	-0.0933	0.0003	-0.147	-0.0396	True
macrobase	micoes	0.0335	0.3774	-0.0202	0.0872	False

(b) Tukey's HDS on the algorithms average explanation time

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	-1.2475	0.9987	-14.7715	12.2765	False
coin	exstream	-1.675	0.996	-15.199	11.849	False
coin	macrobase	25.5602	0.0001	12.0363	39.0842	True
coin	micoes	-1.3323	0.9984	-14.8563	12.1917	False
exos	exstream	-0.4275	1.0	-13.9515	13.0965	False
exos	macrobase	26.8078	0.0	13.2838	40.3317	True
exos	micoes	-0.0848	1.0	-13.6088	13.4392	False
exstream	macrobase	27.2352	0.0	13.7113	40.7592	True
exstream	micoes	0.3427	1.0	-13.1813	13.8667	False
macrobase	micoes	-26.8925	0.0	-40.4165	-13.3686	True

Fig. 12. The Tukey's HDS test results on the algorithms' performance when window sizes vary.

Non-concurrent data streams. COIN, EXstream, Macrobase, and MICOES algorithms explain outliers in each data stream independently, without the necessity to process data points from all streams collectively. Consequently, they can be applied directly to m data streams by running m instances of the algorithms. On the contrary, EXOS utilizes cross-correlation between data streams to estimate normal values of outliers, necessitating data points from all streams to be processed together in its Estimator component. To showcase the performance of these algorithms on non-concurrent data streams, we run the Synthetic 2 dataset as non-concurrent data streams. We set the window size to ten minutes and simulated each data point in S_1, S_2, S_3 , and S_4 to arrive every 1, 2, 4, and 3 s, respectively. The performance results are reported in Table 5.

In Table 5, it is evident that EXOS stands out as the sole algorithm with average precision, average recall, and average F1 scores surpassing 0.9. While EXstream attains the highest average precision, it concurrently records the lowest average recall. Conversely, Macrobase achieves the perfect average recall but at the expense of the lowest average precision. In this particular scenario, the algorithms are ranked from the highest to the lowest average F1 Score as follows: EXOS, EXstream, MICOES, COIN, and Macrobase. In terms of average explanation time in this scenario, the ranking from fastest to slowest is as follows: EXstream, EXOS, MICOES, COIN, and Macrobase.

7.2.2. The impact of the window size and data points' arrival rate

The number of data points within a window is contingent on both the window size and the arrival rate of data points. Using the Synthetic dataset, which represents four data streams, we explore how the window size affects algorithm performance. We vary the window size from ten to sixty minutes, simulating each data point to arrive every five seconds. Fig. 11 describes the average F1 scores of the algorithms. Notably, EXOS consistently achieves the highest score irrespective of window size. However, with an increase in the window size, its F1 scores show a slight decrease. This suggests that, in EXOS, a higher proportion of inliers compared to outliers does not necessarily lead to a more accurate explanation of outlying attributes. Among all algorithms, Macrobase has the highest average explanation time. The explanation time for Macrobase increases as the window size expands. In contrast, the average explanation time for the remaining algorithms remains relatively low.

We also explore whether the performance of the algorithms' performance significantly differs with varying window sizes. Conducting the One-Way ANOVA test on the average F1 score produces a notably small p -value (less than 0.05), prompting the rejection of the null hypothesis which suggests no difference among the means of algorithms' average F1 Scores. Recognizing statistically significant variations between the means of the algorithms' average F1 scores, we employ Tukey's HDS to identify significant differences among their means.

In Fig. 12(a), the difference in average F1 scores between COIN (group 1) and EXOS (group 2) is 0.4487, indicating that EXOS's mean exceeds COIN's. Recall that the null hypothesis for the Tukey's HDS test is "there are no statistically significant differences between the means of the algorithms' average F1 score". With the adjusted p -value of 0.0000 (shown as 0.0 in Fig. 12) and the

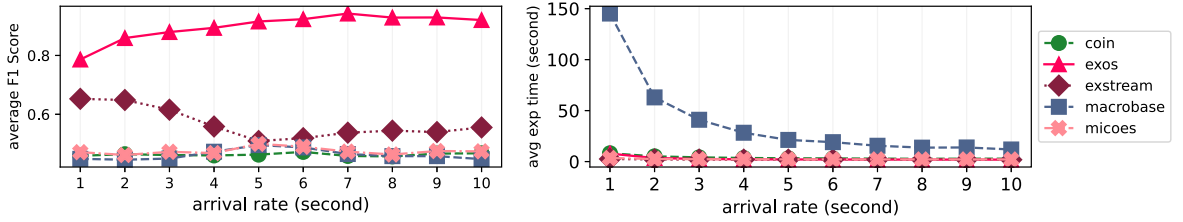


Fig. 13. The impact of the arrival rate of data points on the performance of the algorithms.

Varying arrival rate, fixed window size

(a) Tukey's HDS on the algorithms average F1 Score

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	0.4349	-0.0	0.3933	0.4764	True
coin	exstream	0.1051	0.0	0.0635	0.1466	True
coin	macrobase	-0.0002	1.0	-0.0418	0.0414	False
coin	micoes	0.0119	0.9246	-0.0296	0.0535	False
exos	exstream	-0.3298	-0.0	-0.3714	-0.2882	True
exos	macrobase	-0.4351	-0.0	-0.4766	-0.3935	True
exos	micoes	-0.4229	-0.0	-0.4645	-0.3814	True
exstream	macrobase	-0.1053	0.0	-0.1468	-0.0637	True
exstream	micoes	-0.0931	0.0	-0.1347	-0.0516	True
macrobase	micoes	0.0121	0.9202	-0.0294	0.0537	False

(b) Tukey's HDS on the algorithms average explanation time

group1	group2	meandiff	p-adj	lower	upper	reject
coin	exos	-1.0568	0.9999	-24.4646	22.351	False
coin	exstream	-1.8923	0.9994	-25.3001	21.5155	False
coin	macrobase	33.4102	0.0018	10.0024	56.818	True
coin	micoes	-1.5273	0.9997	-24.9351	21.8805	False
exos	exstream	-0.8355	1.0	-24.2433	22.5723	False
exos	macrobase	34.467	0.0012	11.0592	57.8748	True
exos	micoes	-0.4705	1.0	-23.8783	22.9373	False
exstream	macrobase	35.3025	0.0009	11.8947	58.7103	True
exstream	micoes	0.365	1.0	-23.0428	23.7728	False
macrobase	micoes	-34.9375	0.001	-58.3453	-11.5297	True

Fig. 14. The Tukey's HDS test results on the algorithms' performance when arrival rates vary.

confidence interval excluding, it is plausible to reject the null hypothesis. However, there is insufficient evidence to reject the null hypothesis for three comparisons: COIN vs Macrobase, COIN vs MICOES, and Macrobase vs MICOES. These results suggest similar average F1 scores among COIN, Macrobase, and MICOES despite varying window sizes. Notably, when comparing EXOS (group 1) with EXstream, Macrobase, and MICOES (group 2), the negative mean difference values indicate that EXOS's average F1 score surpasses those of these algorithms.

In Fig. 12(b), only four comparisons support rejecting the null hypothesis. These comparisons are COIN vs Macrobase, EXOS vs Macrobase, EXstream vs Macrobase, and Macrobase vs MICOES. This outcome confirms that only Macrobase has a higher average explanation time, while the other algorithms have similar time performance when the arrival rate is five seconds, and window sizes vary between ten to sixty minutes.

Utilizing the same dataset, we investigate the influence of the data point's arrival rate by altering it between one and ten seconds, with a fixed window size of thirty minutes. The outcomes are depicted in Figs. 13 and 14. The arrival rate signifies the time interval to await a new data point after the preceding one has arrived. For instance, an arrival rate of 10 s implies a data point arrives every ten seconds. With an increasing arrival rate, there is a reduction in the number of data points within the fixed-size window.

Fig. 13 illustrates that EXOS attains the highest average F1 score across all variations of the arrival rate. This finding supports the notion that, for achieving a superior average F1 score, EXOS does not necessitate an increase in data points but rather requires a better understanding of the local context for each outlier. More data points do not inherently translate to an improved local context. Notably, Macrobase demonstrates the lengthiest average explanation time in all arrival rate variations. However, as it takes longer for a new data point to arrive, the average explanation time is going down. This can be attributed to the decrease in the number of data points within the window as it takes longer for a data point to arrive, resulting in a shorter average explanation time.

In Fig. 14(a), only three pairwise comparisons COIN vs. Macrobase, COIN vs. MICOES, and COIN vs. Macrobase vs. MICOES, suggest no significant mean difference in the average F1 score between the algorithms. This aligns with the observations from Fig. 13 (a) showing COIN, Macrobase, and MICOES exhibit comparable average F1 scores. Fig. 14(b) further validates that Macrobase exhibits significantly higher mean differences in average explanation time compared to other algorithms. Additionally, it reveals no significant mean differences in the average explanation time of COIN, EXOS, EXstream, and Macrobase.

7.2.3. The impact of concept drift

We investigate the impact of data distribution changes, known as concept drift, on the performance of the algorithms. We simulate four data streams, each having ten attributes, eleven windows, 1K data points per window, and the same arrival rate. Our experiments focus on two key questions: (Q1) Does the performance of the algorithms alter when a concept drift occurs in a specific window?, and (Q2) How does the frequency of concept drift occurrences affect the algorithms' performance?

To address Question (Q1), we simulate the synthetic datasets with varying locations of concept drift occurrences, i.e., varying the IDs of the windows where concept drifts occur. Following the similar approaches used to verify that concept drifts indeed occur in

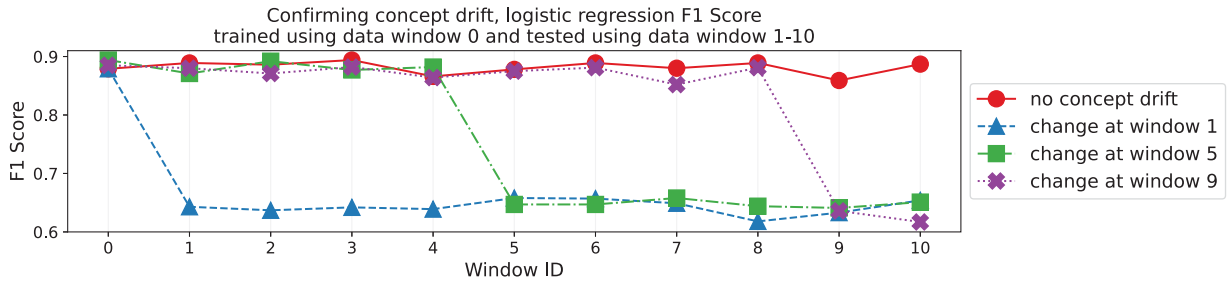


Fig. 15. Using a static data-based classifier to confirm whether a dataset indeed has concept drift.

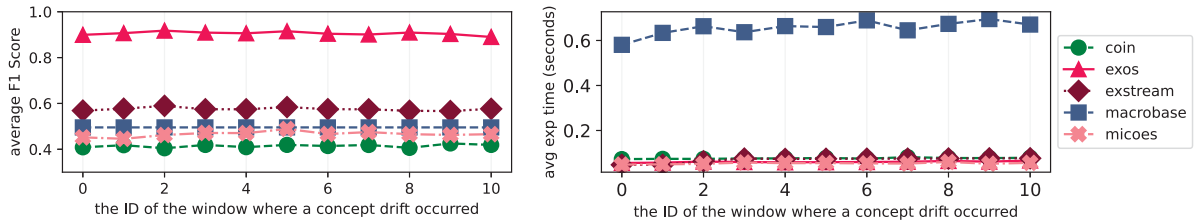


Fig. 16. The impact of the location of concept drift on the algorithms' average F1 Score and explanation time.

a dataset [39,59], we confirm the presence of concept drift by adding binary classification labels to the datasets and then training a logistic regression classifier using the data in Window 0 before testing it using the data in Windows 1 through 10. It is worth noting that this classification model is designed for static data and thus is expected to perform similarly across all the windows in the absence of concept drifts. However, as shown in Fig. 15, once there is a change in data distribution at Window i , the classifier performance begins to decline, indicating the presence of concept drifts in the datasets.

Fig. 16 demonstrates the performance of the outlying attribute algorithms when the location of a concept drift occurrence is varied from Window 0 to Window 10. EXOS achieves the highest average F1 scores, followed by EXstream, MacroBase, MICOES, and COIN. However, MacroBase has the highest average explanation time, while other algorithms performed similarly. The consistent performance is maintained by all the algorithms regardless of the concept drift locations as their model and/or temporal context are updated in each window. Thus, we can conclude that the performance of the algorithms is unaffected by the locations of concept drifts.

To answer Question (Q2), we vary the occurrences of concept drifts across eleven windows ranging from 0% to 100%. A 0% concept drift indicates that Windows 1–10 have the same data distribution as Window 0. A 50% concept drift means from Window 1 through Window 10; there are five windows where data distribution changes. A 100% concept drift means the data distribution changes every time the window slides. We run experiments on four synthetic datasets. Sets 1, 2, and 3 are composed of multivariate Gaussian data. In Set 1, a change in the mean vectors occurs during specific windows while the covariance matrices remain unchanged. In contrast, in Set 2, only the covariance matrices change while the mean vectors stay constant. Finally, both the mean vectors and covariance matrices change simultaneously in Set 3.

Set 4 has mixed data distributions consisting of randomly chosen multivariate Gaussian, Beta, Gamma, and Exponential distributions. For instance, when we set the percentage concept drifts to be 50%; Windows 0–2 can have a Gaussian distribution, while Window 3 may use a gamma distribution. Likewise, Windows 4–7 could be exponential, Window 8 beta and finally, Windows 9–10 are gamma distributed.

Fig. 17 displays the performance of the algorithms on Sets 1–4, with EXOS having the best average F1 score. All the algorithms have similar average explanation times while MacroBase has the highest one. The frequency of concept drifts hardly has any effect on the performance of EXOS, EXstream, Coin, and MacroBase. With EXOS, the eigenvectors used to estimate the normal values of the outliers in a window are updated when the window slides. The same update applies to its temporal context component, which, together with the estimated normal values of the outliers, forms the inlier and outlier classes. Therefore, even when a window data distribution differs from that in the previous window, the EXOS components are constantly adjusted, allowing it to maintain its performance. A similar window-sliding update on the explanation model also applies to the other algorithms. Even though COIN is intended for static data, we run it in batches such that the subsets of data points used to build its outlier explanation model are continually updated as the window slides. The average F1 score slightly fluctuates on MICOES, indicating that the temporal context formed using denstream-based micro-clusters in a window still mixes with some micro-clusters from the previous window. MICOES's average explanation time also slightly increases in Sets 2–4 when the frequency of concept drifts is getting higher. This explanation time increase relates to the maintenance of its micro-clusters. When the data distribution changes more often, the state of its micro-clusters also changes, requiring more time to maintain/update.

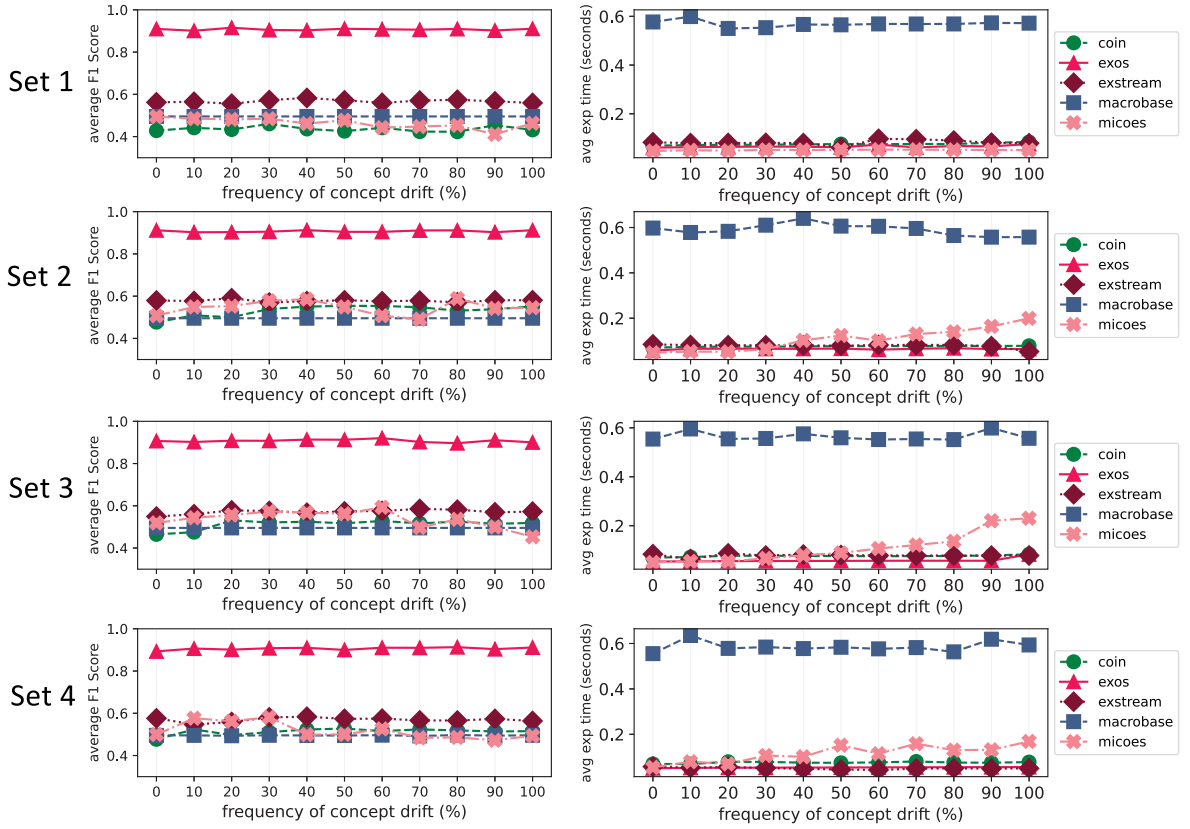


Fig. 17. The impact of the location of concept drift on the algorithms' average F1 Score and explanation time.

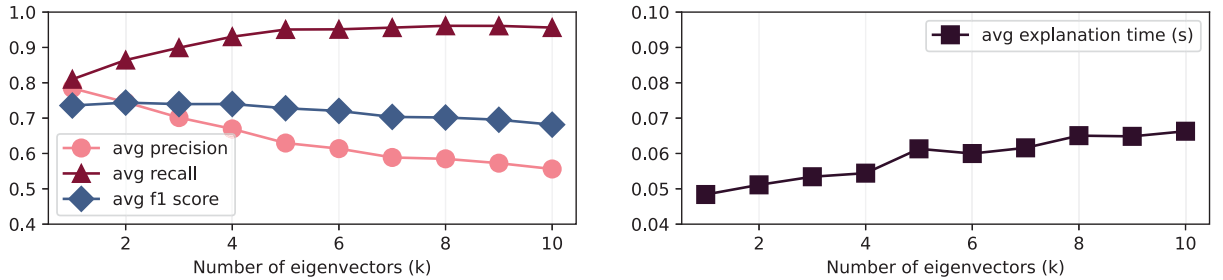


Fig. 18. The impact of the number of eigenvectors (k) on performance of EXOS.

7.2.4. The impact of EXOS-specific parameters

We study the impact of two EXOS user-defined parameters using the synthetic dataset where the window size and arrival rate of each stream are set to be the same such that each window contains 1000 data points.

Impact of the number of eigenvectors (k). To generate the estimated normal value of each outlier found in each window, EXOS needs the estimated eigenvectors (principal components) whose size depends on k. k can be varied from 1 to D (the total number of attributes). In our study, we vary k from 1 to 10. Fig. 18 tells us that k has an impact on the average precision, recall, and F1 score. As k increases, the average recall also increases in average by 1.89% but the average precision and F1 score decrease in average by 3.73% and 0.84% respectively. Unsurprisingly, as k gets larger, the average explanation time gets longer in average by 3.6% as the EXOS' estimation component uses more eigenvectors. Therefore, to provide the explanations quickly, a small value of k such as k = 1 would be preferred (see Fig. 19).

Impact of the outlier attribute contribution threshold γ . The threshold γ guides EXOS in identifying attributes considered as the outlying attributes of an outlier after the generating decision boundary that separates the inlier class from the outlier class for that outlier. $\gamma = 0.01$ indicates that only attributes that have the outlying contribution scores greater than 1% are considered the outlying

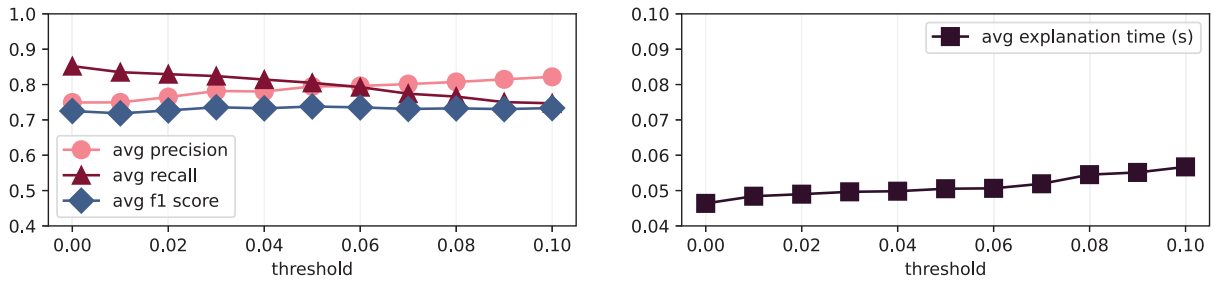


Fig. 19. The impact of the outlier attribute contribution threshold (γ) on performance of EXOS.

attributes. Fig. 19 demonstrates the average precision slightly increases in average by 1% as γ increases by 0.01. A higher γ value facilitates the reduction of False Positive outlying attributes. Conversely, the average recall experiences a slight decline in average by 1% with increasing γ , as higher γ values contribute to more False Negative outlying attributes. Nevertheless, the average F1 score remains steady as the reduction of False Positive is counterbalanced by the increase in False Negative. Additionally, as the γ value increases by 0.01, the average explanation time increases by around 2%.

8. Conclusion and future research direction

This paper presents EXOS, an algorithm to discover outliers' outlying attributes in real-time data streams that considers the characteristics of data streams. To address the challenge of the continuous arrival and unbounded volume of data streams (the notion of infinity), EXOS employs time-based tumbling windows where data points reside in memory until they expire, replaced by a new set of data points. To ensure timely processing of the data before they leave the active tumbling window, EXOS updates the eigenvectors or principal components model that captures the cross-correlation of data streams and clusters data points of each data stream in parallel. Both processes involve single-pass computation on the data points.

To capture the cross-correlation among data streams, EXOS's Estimator component needs to update the principal component model using data points from all active windows of the data streams. However, the data streams can have the same arrival rates (concurrent data streams) or varying arrival rates (non-concurrent data streams). EXOS handles concurrent or non-concurrent data streams by aligning data points in the active windows based on their timestamp steps. It then selectively employs aligned data corresponding to the timestamp steps that have data points from all streams, to update the principal component model.

EXOS determines the outlying attributes of each outlier by forming a local context using the principal components model and the closest inlier cluster. Adaptability to changes in data distribution, known as concept drift, is inherent in EXOS, as the model and the clusters are updated whenever a new window slides.

Experimental results demonstrate that EXOS is a promising technique for discovering outlying attributes of outliers in real-time data streams. It is shown to be effective when applied to data streams with both identical and varying arrival rates, achieving the best average F1 score (with one exception) while maintaining a competitive average explanation time. Overall, on average EXOS achieves 45.6% higher average F1 Score. EXOS has 1.3 to 29.6 times faster average explanation time than the other algorithms except when Synthetic 2 Dataset is simulated as non-concurrent data streams where EXstream is 1.2 times faster.

For future research, we aim to extend EXOS to address additional characteristics of data streams, specifically transiency and uncertainty. Transiency refers to the diminishing importance of data points over time, raising the question of how to incorporate this factor into the discovery of an outlier's outlying attributes. Currently, EXOS treats all data points in the active window with equal importance, but the significance of a data point may vary based on its temporal proximity to an outlier. Simultaneously, addressing uncertainty in data streams arising from errors or environmental factors presents an intriguing avenue for further research, especially regarding its impact on the accuracy of outliers' outlying attributes.

CRedit authorship contribution statement

Egawati Panjei: Writing – review & editing, Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Le Gruenwald:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have shared our/data on Github. The Github Link is available in the paper.

References

- [1] G. Krempf, I. Žliobaite, D. Brzeziński, E. Hüllermeier, M. Last, V. Lemaire, T. Noack, A. Shaker, S. Sievi, M. Spiliopoulou, J. Stefanowski, Open challenges for data stream mining research, *SIGKDD Explor. Newsl.* 16 (2014) 1–10, <http://dx.doi.org/10.1145/2674026.2674028>, URL <https://doi.org.ezproxy.lib.ou.edu/10.1145/2674026.2674028>.
- [2] M.S. Sadik, L. Gruenwald, Research issues in outlier detection for data streams, *SIGKDD Explor.* 15 (2014) 33–40.
- [3] S. Yoon, Y. Shin, J.G. Lee, B.S. Lee, Multiple dynamic outlier-detection from a data stream by exploiting duality of data and queries, in: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2021, <http://dx.doi.org/10.1145/3448016.3452810>.
- [4] A. Milenkoski, M. Vieira, S. Kounev, A. Avritzer, B.D. Payne, Evaluating computer intrusion detection systems: A survey of common practices, *ACM Comput. Surv.* 48 (2015) <http://dx.doi.org/10.1145/2808691>.
- [5] D.J. Weller-Fahy, B.J. Borghetti, A.A. Sodemann, A survey of distance and similarity measures used within network intrusion anomaly detection, *IEEE Commun. Surv. Tutor.* 17 (2015) 70–91.
- [6] A.M. Nancy, G.S. Kumar, S. Veena, N.A. Vinoth, M. Bandyopadhyay, Fraud detection in credit card transaction using hybrid model, 2277, 2020, <http://dx.doi.org/10.1063/5.0025561>,
- [7] N. Rtayli, N. Enneya, Enhanced credit card fraud detection based on SVM-recursive feature elimination and hyper-parameters optimization, *J. Inf. Secur. Appl.* 55 (2020) <http://dx.doi.org/10.1016/j.jisa.2020.102596>.
- [8] S. Saraf, A. Phakatkar, Detection of credit card fraud using a hybrid ensemble model, *Int. J. Adv. Comput. Sci. Appl.* 13 (2022) <http://dx.doi.org/10.14569/IJACSA.2022.0130953>.
- [9] M. Salehi, C. Leckie, J.C. Bezdek, T. Vaithianathan, X. Zhang, Fast memory efficient local outlier detection in data streams, *IEEE Trans. Knowl. Data Eng.* 28 (2016) <http://dx.doi.org/10.1109/TKDE.2016.2597833>.
- [10] C. O'Reilly, A. Gluhak, M.A. Imran, S. Rajasegarar, Anomaly detection in wireless sensor networks in a non-stationary environment, *IEEE Commun. Surv. Tutor.* 16 (2014) <http://dx.doi.org/10.1109/SURV.2013.112813.00168>.
- [11] N. Shahid, I.H. Naqvi, S.B. Qaisar, Characteristics and classification of outlier detection techniques for wireless sensor networks in harsh environments: a survey, *Artif. Intell. Rev.* 43 (2015) <http://dx.doi.org/10.1007/s10462-012-9370-y>.
- [12] G. Ditzler, M. Roveri, C. Alippi, R. Polikar, Learning in nonstationary environments: A survey, *IEEE Comput. Intell. Mag.* 10 (2015) <http://dx.doi.org/10.1109/MCI.2015.2471196>.
- [13] H.M. Gomes, J. Read, A. Bifet, J.P. Barddal, J. Gama, Machine learning for streaming data: State of the art, challenges, and opportunities, *SIGKDD Explor. Newsl.* 21 (2019).
- [14] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, *IEEE Trans. Knowl. Data Eng.* 31 (2019) <http://dx.doi.org/10.1109/TKDE.2018.2876857>.
- [15] S. Wang, L.L. Minku, X. Yao, A systematic study of online class imbalance learning with concept drift, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (2018) <http://dx.doi.org/10.1109/TNNLS.2017.2771290>.
- [16] K. Yu, W. Shi, N. Santoro, Designing a streaming algorithm for outlier detection in data mining - an incremental approach, *Sensors (Switzerland)* 20 (2020) <http://dx.doi.org/10.3390/s20051261>.
- [17] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, *ACM Comput. Surv.* 41 (2009) 15:1–15:58.
- [18] E. Panjei, L. Gruenwald, E. Leal, C. Nguyen, S. Silvia, A survey on outlier explanations, *The VLDB J.* 31 (2022) 977–1008, <http://dx.doi.org/10.1007/s00778-021-00721-1>.
- [19] L.A. Bull, K. Worden, E.J. Cross, N. Dervilis, Outlier ensembles : an alternative robust method for inclusive outlier analysis with structural health monitoring data, 2018.
- [20] E.P. Carden, P.J. Fanning, Vibration based condition monitoring: A review, *Struct. Health Monit.* 3 (2004) 355–377.
- [21] K. Worden, G. Manson, The application of machine learning to structural health monitoring, *Phil. Trans. R. Soc. A* 365 (2006) 515–537.
- [22] M.A. Siddiqui, A. Fern, T.G. Dietterich, W.-K. Wong, Sequential feature explanations for anomaly detection, *ACM Trans. Knowl. Disc. Data (TKDD)* 13 (2019) 1–22.
- [23] C. Bigoni, J.S. Hesthaven, Simulation-based anomaly detection and damage localization: an application to structural health monitoring, *Comput. Methods Appl. Mech. Engrg.* 363 (2020) <http://dx.doi.org/10.1016/j.cma.2020.112896>.
- [24] X.-H. Dang, B. Mícenková, I. Assent, R.T. Ng, Local outlier detection with interpretation, in: *ECML/PKDD*, 2013.
- [25] F. Keller, E. Müller, A. Wixler, K. Böhm, Flexible and adaptive subspace search for outlier analysis, in: *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management*, 2013.
- [26] B. Mícenková, R.T. Ng, X.-H. Dang, I. Assent, Explaining outliers by subspace separability, in: *2013 IEEE 13th International Conference on Data Mining*, 2013, pp. 518–527.
- [27] M. Macha, L. Akoglu, Explaining anomalies in groups with characterizing subspace rules, *Data Min. Knowl. Discov.* 32 (2018) 1444–1480.
- [28] N. Liu, D. Shin, X. Hu, Contextual outlier interpretation, in: *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*, 2018, pp. 2461–2467.
- [29] N. Gupta, D. Eswaran, L. Akoglu, C. Faloutsos, N. Shah, Beyond outlier detection: LookOut for pictorial explanation, in: *ECML PKDD*, 11051, 2019, http://dx.doi.org/10.1007/978-3-030-10925-7_8.
- [30] H. Zhang, Y. Diao, A. Meliou, EXstream: Explaining anomalies in event stream monitoring, in: *International Conference on Extending Database Technology, EDBT*, 2017.
- [31] F. Abuzaid, P. Bailis, J. Ding, E. Gan, S. Madden, D. Narayanan, K. Rong, S. Suri, MacroBase: Prioritizing attention in fast data, *ACM Trans. Database Syst.* 43 (2018) <http://dx.doi.org/10.1145/3276463>.
- [32] F. Song, Y. Diao, J. Read, A. Stiegler, A. Bifet, EXAD: A system for explainable anomaly detection on big data traces, in: *2018 IEEE International Conference on Data Mining Workshops, ICDMW*, 2018, pp. 1435–1440.
- [33] E. Panjei, L. Gruenwald, E. Leal, C. Nguyen, Micro-clusters-based outlier explanations for data streams, in: *The 1st Workshop on Anomaly and Novelty Detection, Explanation and Accommodation (ANDEA)*, co-located with 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2021, pp. 1–8, URL <https://sites.google.com/view/andea2021/accepted-papers>.
- [34] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R.J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, S. Whittle, The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing, 8, 2015, <http://dx.doi.org/10.14778/2824032.2824076>,
- [35] E. Panjei, L. Gruenwald, EXOS: Explaining outliers in data streams, in: R. Wrembel, J. Gamper, G. Kotsis, A.M. Tjoa, I. Khalil (Eds.), *Big Data Analytics and Knowledge Discovery, DaWaK 2023* ed., in: *Lecture Notes in Computer Science*, vol. 14148, Springer Nature Switzerland, Cham, 2023, pp. 25–41, http://dx.doi.org/10.1007/978-3-031-39831-5_3.
- [36] M. Kopp, T. Pevný, M. Holena, Interpreting and clustering outliers with sapling random forests, in: *ITAT*, 2014.
- [37] N. Takeishi, Shapley values of reconstruction errors of PCA for explaining anomaly detection, in: *2019 International Conference on Data Mining Workshops, ICDMW*, 2019, pp. 793–798.

- [38] P. Carbone, M. Fragkoulis, V. Kalavri, A. Katsifodimos, Beyond analytics: The evolution of stream processing systems, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 2651–2658, <http://dx.doi.org/10.1145/3318464.3383131>.
- [39] S. Das, Best practices for dealing with concept drift, 2023, URL <https://neptune.ai/blog/concept-drift-best-practices>.
- [40] S. Wang, L.L. Minku, N. Chawla, X. Yao, Learning in the presence of class imbalance and concept drift, *Neurocomputing* 343 (2019) <http://dx.doi.org/10.1016/j.neucom.2019.01.080>.
- [41] V. Jacob, F. Song, A. Stiegler, B. Rad, Y. Diaio, N. Tatbul, Exathlon: A benchmark for explainable anomaly detection over time series, *Proc. VLDB Endow.* 14 (2021) 2613–2626, <http://dx.doi.org/10.14778/3476249.3476307>.
- [42] J. Shlens, A tutorial on principal component analysis, 2014, CoRR abs/1404.1100 URL <http://arxiv.org/abs/1404.1100>.
- [43] I.T. Jolliffe, J. Cadima, Principal component analysis: A review and recent developments, *Phil. Trans. R. Soc. A* 374 (2016) <http://dx.doi.org/10.1098/rsta.2015.0202>.
- [44] C.L. Li, H.T. Lin, C.J. Lu, Rivalry of two families of algorithms for memory-restricted streaming PCA, in: Proceedings of the 19th International Conference on Artificial Intelligence and Statistics, AISTATS, 2016.
- [45] W.M. Mendenhall, T.L. Sincich, Chapter 2 descriptive statistics, sixth ed., 2016, pp. 25–75.
- [46] B.E. Boser, I.M. Guyon, V.N. Vapnik, Training algorithm for optimal margin classifiers, in: Proceedings of the Fifth Annual ACM Workshop on Computational Learning Theory, 1992, <http://dx.doi.org/10.1145/130385.130401>.
- [47] R. Tibshirani, Regression shrinkage and selection via the lasso: A retrospective, *J. R. Stat. Soc. Ser. B Stat. Methodol.* 73 (2011) <http://dx.doi.org/10.1111/j.1467-9868.2011.00771.x>.
- [48] M.J. Zaki, W.M. J.R., Representative-based clustering, in: *Data Mining and Analysis Fundamental Concepts and Algorithms*, Cambridge University Press, 2014, pp. 334–335.
- [49] W.H. Press, S.A. Teukolsky, W.T. Vetterling, B.P. Flannery, 2.10 QR decomposition, in: *Numerical Recipes the Art of Scientific Computing*, third ed., 2007, pp. 102–103, URL <https://numerical.recipes/book.html>.
- [50] D.B. Thomas, W. Luk, Multivariate Gaussian random number generation targeting reconfigurable hardware, *ACM Trans. Reconfigurable Technol. Syst.* 1 (2008) <http://dx.doi.org/10.1145/1371579.1371584>.
- [51] J. Price, A. Wong, T. Yuan, J. Mathews, T. Olorunniwo, Stochastic gradient descent, in: Cornell University Computational Optimization Open Textbook - Optimization Wiki, 2020, URL http://optimization.cbe.cornell.edu/index.php?title=Stochastic_gradient_descent.
- [52] S. Learn, Support vector machines, 2024, scikit-learn.org URL <https://scikit-learn.org/stable/modules/svm.html#svm>.
- [53] P. Bodik, W. Hong, C. Guestrin, S. Madden, M. Paskin, R. Thibaux, Intel Lab Data, DesignSafe-CI, 2004, URL <http://db.csail.mit.edu/labdata/labdata.html>.
- [54] T. Buckreis, A. Winders, P. Wang, S. Brandenberg, J. Stewart, Microtremor Data Collected in Sacramento-San Joaquin Delta Region of California, DesignSafe-CI, 2021, <http://dx.doi.org/10.17603/ds2-dk6t-8610>.
- [55] S. Makonin, AMPds2: The Almanac of Minutely Power dataset (Version 2), Harvard Dataverse, 2016, <http://dx.doi.org/10.7910/DVN/FIE0S4>.
- [56] J. Hardin, S.R. Garcia, D. Golan, A method for generating realistic correlation matrices, *Ann. Appl. Stat.* 7 (2013) 1733–1762.
- [57] W.M. Mendenhall, T.L. Sincich, Chapter 14 The Analysis of Variance for Designed Experiments, sixth ed., 2016, pp. 742–836.
- [58] H. Abdi, L.J. Williams, Tukey's honestly significant difference (HSD) test, in: N. Salkind (Ed.), *Encyclopedia of Research Design*, Sage, Thousand Oaks, CA, 2010.
- [59] F. Gu, Concept Drift Detection for Machine Learning with Stream Data, University of Technology Sydney, 2019, URL <https://opus.lib.uts.edu.au/bitstream/10453/140165/2/02whole.pdf>.



Egawati Panjei earned her Ph.D. and MS in Computer Science from The University of Oklahoma and BS in Computer Science from Institut Teknologi Sepuluh Nopember. Her research interests include Machine Learning, Data Mining, and Data Stream Management. She is an Assistant Professor at the Department of Computer Science, Lyle College of Engineering, Southern Methodist University. Her experience as a Software Engineer enriches her practical insights into software development and project management. Additionally, she was a recipient of the Fulbright Scholarship for her master's degree.



Le Gruenwald is a Professor, Dr. David W. Franke Professor, and Samuel Roberts Noble Foundation Presidential Professor in the School of Computer Science at The University of Oklahoma. She received her Ph.D. in Computer Science from Southern Methodist University, MS in Computer Science from the University of Houston, and BS in Physics from the University of Saigon. She worked for National Science Foundation as a Cluster Lead and Program Director of the Information Integration and Informatics program and a Program Director of the Cyber Trust program. Dr. Gruenwald's major research interests include Data Management, Data Mining, and Information Privacy and Security.